



Universidad de Extremadura

Escuela Politécnica

**Máster Universitario en Ingeniería
de Telecomunicación**

Trabajo Fin de Máster

**Sistema de Compostaje Inteligente Utilizando
MQTT y FIWARE**

**Ignacio Diestro Gil
Julio 2025**



Universidad de Extremadura

Escuela Politécnica

**Máster Universitario en Ingeniería
de Telecomunicación**

Trabajo Fin de Máster

**Sistema de Compostaje Inteligente Utilizando
MQTT y FIWARE**

Autor: Ignacio Diestro Gil

Tutor: Antonio Gordillo Guerrero

Agradecimientos

Para comenzar, me gustaría darle las gracias a mi tutor, Antonio Gordillo Guerrero. Por contar conmigo para cada proyecto y por brindarme siempre su apoyo y confianza desde el primer día. Su disposición, cercanía y facilidad con la que hace todo han facilitado enormemente el desarrollo del trabajo.

También quiero agradecer a Jorge Osuna Pons, especialista en FIWARE de la Diputación de Badajoz, por su tiempo, dedicación y disposición a la hora de ayudarme a conocer y utilizar la plataforma. Su acompañamiento ha sido clave para comprender mejor la tecnología y aplicarla de forma efectiva en este proyecto.

Finalmente, me gustaría darle las gracias a un buen amigo que estuvo conmigo desde mis primeras andaduras en este mundo del desarrollo y calidad de software. Carlos Delgado Guiberteau, gracias por ser guía, luz y sobre todo amigo en este camino que no empezó como una tarea fácil. Gracias por las tardes interminables ayudándome a corregir código, configurar redes y a planificar gran parte de mi TFG. Desde luego, todo se habría hecho más cuesta arriba sin tu ayuda y estoy seguro de que no habría alcanzado los estándares de calidad que considero se consiguieron en aquel momento. Espero algún día poder devolverte todo lo que me ayudaste y todo lo que lo sigues haciendo.

Y, para terminar, muchas gracias a Irene del Carmen López del Reino, porque no todo el apoyo que se necesita se encuentra en códigos, programas y configuraciones, a veces, una charla entre amigos es incluso más productiva.

Resumen

En un contexto global marcado por la creciente preocupación medioambiental y la necesidad de optimizar los procesos de gestión de residuos, este Trabajo Fin de Máster propone el diseño e implementación de un sistema de control y monitorización de composteras, es decir, biodigestores aeróbicos de materia orgánica inteligentes mediante tecnologías de telecomunicación de amplia implantación. El objetivo principal es facilitar el seguimiento y análisis de variables críticas del proceso de compostaje, como temperatura, humedad, gases o generación de lixiviados, mediante sensores conectados a través del protocolo MQTT y un sistema centralizado que recopila, procesa y envía la información a una plataforma basada en el estándar Fiware.

El sistema está orientado a entornos rurales o instalaciones locales que requieren soluciones de bajo coste y fácil despliegue, definimos para ello un sistema embebido modular basado en microcontroladores. Se evalúan diferentes opciones de infraestructura de comunicación, desde brokers MQTT profesionales hasta la integración con sistemas de información más amplios. Asimismo, se presta atención especial a la escalabilidad y la capacidad de integración futura con servicios de análisis o visualización de datos.

Este proyecto se enmarca dentro de una iniciativa de colaboración entre la Escuela Politécnica de Cáceres de la Universidad de Extremadura y la Diputación de Badajoz, que proporciona apoyo en el uso de tecnologías Fiware, y busca demostrar la viabilidad técnica de un sistema modular y replicable en diferentes contextos de gestión de residuos orgánicos.

Los desarrollos dentro de este proyecto, servirán como modelo base de prueba de concepto para futuras iteraciones impulsadas por el *Fiware Space* de la Diputación de Badajoz.

Abstract

In a global context marked by growing environmental concerns and the need to optimize waste management processes, this Master's Thesis proposes the design and implementation of a control and monitoring system for smart composters using IoT technologies and the Fiware Protocol. The main objective is to facilitate the tracking and analysis of critical variables in the composting process, such as temperature, humidity, or leachate generation, through sensors connected via the MQTT protocol and a centralized system that collects, processes, and sends the information to a FIWARE-based platform.

The system is aimed at rural environments or local facilities that require low-cost, easy-to-deploy solutions, considering the possibility of using microcontroller based devices. Various infrastructure options are evaluated, from professional MQTT brokers to integration with broader information systems. Special attention is also given to scalability and the potential for future integration with data analysis or visualization services.

This project is part of a collaborative initiative between School of Tecnology of University of Extremadura and the Diputación de Badajoz, which provides support in the use of FIWARE technologies, and seeks to demonstrate the technical feasibility of a modular and replicable system for different organic waste management contexts.

Índice General

1	Introduction	1
1.1	Project scope.....	2
1.1	Motivation	4
2	Objectives	5
3	State of the Art.....	7
3.1	Organic waste management and composting	7
3.1.1	Introduction	7
3.1.2	Composting basics.....	8
3.1.3	Different types of composting	8
3.1.4	Advantages of composting	10
3.2	IoT technologies in agriculture and environmental management	12
3.2.1	Introduction	12
3.2.2	Smart agriculture	12
3.3	IoT communication protocols.....	13
3.3.1	Introduction	13
3.3.2	IoT communication architectures	13
3.3.3	Main IoT protocols	14
3.3.4	MQTT communication structure	18
3.4	Existing smart composting solutions.....	20
3.4.1	Introduction	20
3.4.2	Composting in Extremadura.....	20
3.4.3	Proposed solution	21
4	Metodología.....	23
4.1	Fases del proyecto	24
4.1.1	Iteración 1	24
4.1.2	Iteración 2	24
4.1.3	Iteración 3	25
4.1.4	Iteración 4	25
4.1.5	Iteracion 5	26
4.1.6	Iteración 6	26
4.2	Planificación del proyecto	27
5	Desarrollo e implementación.....	29
4.3	Arquitectura software	29

4.3.1	Diseño general del sistema	29
4.3.2	Tecnologías y lenguajes utilizados	30
4.3.3	Estructura de módulos y servicios	33
4.4	Arquitectura de comunicación	43
4.4.1	Protocolo de comunicación seleccionado	43
4.4.2	Topología de red del sistema	43
4.4.3	Formato de los mensajes intercambiados	44
4.4.4	Broker y gestión de tópicos	44
4.4.5	Diagrama de flujo de datos y comunicación	46
4.5	Arquitectura hardware	47
4.5.1	Descripción de componentes hardware	47
4.5.2	Esquema de conexiones y puertos	54
4.5.3	Consumo energético y autonomía	57
4.5.4	Diagrama de arquitectura hardware	59
4.6	Despliegue y montaje físico	60
4.6.1	Entorno de instalación	60
4.6.2	Montaje e instalación	61
4.6.3	Análisis de costes y tiempo de despliegue	64
6	Resultados	67
7	Conclusions	69
8	Trabajos Futuros	71
9	Bibliografía	73
10	Anexos	79
	Anexo 1	79
	Anexo 2	81
	Anexo 3	86
	Anexo 4	88
	Anexo 5	94
	Anexo 6	95
	Anexo 7	97
	Anexo 8	98
	Anexo 9	100
	Anexo 10	101
	Anexo 11	104
	Anexo 12	105

Anexo 13	108
Anexo 14	110
Anexo 15	111
Anexo 16	112

Índice de Ilustraciones

Ilustración 1: Composting stages [4]	7
Ilustración 2: Organic stages of composting [6]	8
Ilustración 3: Industrial Composting [6]	10
Ilustración 4: Advantages of composting [11]	11
Ilustración 5: Smart agriculture [13]	13
Ilustración 6: Broker-Based protocols (Ex: MQTT) [16]	14
Ilustración 7: Bus-Based protocols [17]	14
Ilustración 8: CoAP protocol [19]	15
Ilustración 9: AMQP protocol [21]	15
Ilustración 10: MQTT protocol [23]	16
Ilustración 11: JMS protocol [25]	16
Ilustración 12: DDS protocol [27]	17
Ilustración 13: XMPP protocol [29]	17
Ilustración 14: MQTT in the OSI model [30]	18
Ilustración 15: Initial connection bróker-client MQTT [30]	18
Ilustración 16: MQTT message structure [31]	18
Ilustración 17: Fixed header structure in MQTT	19
Ilustración 18: Variable header structure in MQTT	19
Ilustración 19: Esquema desarrollo iterativo	23
Ilustración 20: Repositorio GitHub TFM-IDG-SmartComposters	29
Ilustración 21: Función 'setup' del fichero main.cpp	36
Ilustración 22: Función 'loop' del fichero main.cpp	36
Ilustración 23: Sistema Linux y Docker en servidor local	37
Ilustración 24: Ficheros de despliegue en servidor local	37
Ilustración 25: Node-Red, flujo Conexiones MQTT	39
Ilustración 26: Node-Red, flujo UI Control/Dashboards	40
Ilustración 27: Node Red, gráficas de datos	40
Ilustración 28: Node Red, flujo Conexión Fiware	41
Ilustración 29: Diagrama de arquitectura Software	42
Ilustración 30: Tipología de red en estrella del sistema	43
Ilustración 31: Ejemplo formato json para Fiware	44
Ilustración 32: Diagrama de comunicación entre dispositivos	46
Ilustración 33: M5Stack Tough [1]	47
Ilustración 34: Sensor ENV IV [59]	49
Ilustración 35: Sensor ENS160+AHT2X [60]	50
Ilustración 36: Sensor DS12B20 [62]	51
Ilustración 37: Sensor PH0-14 + Sonda [65]	52
Ilustración 38: Raspberry PI 5 [66]	53
Ilustración 39: Puertos de conexión del M5Stack Tough [1]	54
Ilustración 40: Pinout PORT A M5Stack Toug	55
Ilustración 41: Conexiones I2C M5Stack Tough	55
Ilustración 42: Conexión 1Wire M5Stack Tough	56
Ilustración 43: Conexiones analógicas M5Stack Tough	57
Ilustración 44: Diagrama de arquitectura hardware	59

Ilustración 45: Diagrama de montaje general.....	61
Ilustración 46: Distribución de componentes en las composteras.....	62
Ilustración 47: Distribución de conexiones eléctricas	63

Índice de Tablas

Tabla 1: Consumo medio del sistema para 2h de funcionamiento	57
Tabla 2: Autonomía del sistema con batería	58
Tabla 3: Estimación de costes materiales	64
Tabla 4: Estimación de disponibilidad de materiales	65
Tabla 5: Distribución de tiempos para despliegue	65

1 Introduction

The growing global awareness of environmental issues and the urgent need to adopt more sustainable waste management models have driven the development of innovative solutions based on digital technologies. In this context, composting emerges as a key strategy for the treatment of organic waste, especially in rural or semi-urban areas where large-scale industrial solutions are not always viable or economically feasible.

This Master's Thesis proposes the design and implementation of an intelligent system for monitoring the composting process, supported by low cost devices based on microcontrollers, asynchronous communication protocol, and the FIWARE architecture. The system enables real-time collection and analysis of critical composting variables. Data is captured by sensors, centralized, processed, and sent to the FIWARE platform of Diputación de Badajoz, facilitating integration with data analysis and visualization systems.

The solution is particularly aimed at local facilities that require low-cost, easy-to-deploy, and adaptable tools. For this purpose, a microcontroller device [1] has been chosen as the sensing device, along with a robust communication infrastructure based on MQTT and FIWARE. Additionally, its deployment in areas with limited or unstable wireless connectivity is analyzed. Throughout the project, the system's scalability and its potential for replication in different contexts are explored.

The project addresses the growing need for sustainable organic waste management by highlighting the environmental and economic benefits of composting, such as reducing landfill use, cutting greenhouse gas emissions, and producing a valuable soil amendment. Public administrations are increasingly required by national and European regulations to implement effective bio-waste treatment systems, making decentralized composting a strategic priority. This work explores scalable solutions; from small community-level setups to larger municipal systems, capable of adapting to different contexts and resource levels.

The structure of the project following as: introduction, objectives, state of art, methodology, development and implementation, results, conclusions, future work, bibliografy and finally the annexes section where we can find main codes used.

1.1 Project scope

This project focuses on the implementation of a control and monitoring architecture for smart composting systems. It is primarily based on low cost and energy hardware, microservices using Docker, MQTT, FIWARE, and open software, as well as their deployment options.

The project addresses the data acquisition phase using sensors connected to an edge computing device, data centralization through a local server based on Node-RED, and the transmission, processing, and visualization of this information via the Fiware standard and data platform of Diputación de Badajoz [2].

The system includes the following components:

- Sensing of key variables in the composting process, such as temperature, ambient and substrate humidity, gases, and leachate generation.
- Use of the MQTT protocol for efficient data transmission from sensor nodes to the central system.
- Local data centralization using Node-RED and a low cost local server.
- Implementation of local logic on the edge device, including basic actions or alerts triggered by predefined thresholds.
- Integration with the FIWARE platform of Diputación de Badajoz, responsible for storing, processing, and enabling subsequent data analysis. Following the entity protocol defined by European Union [3]
- Design of a modular and scalable architecture, capable of being replicated in different contexts and adapted to other requirements or additional variables.

The following aspects are considered outside the scope of this project:

- Automation of actuators or physical control mechanisms within the composter (such as door opening, leachate release, humidity, etc.).
- Detailed analysis of historical data using advanced techniques such as machine learning, although its future feasibility is considered.
- Development of a more user-friendly graphical interface on microcontroller for composting system operators.
- The actual implementation of the system, which will be carried out at a later stage by Diputación de Badajoz and private companies.

This project has been approached from a practical perspective, aiming to demonstrate a functional system under real-world conditions. The ultimate goal is to provide a foundation for future technical and operational expansions in the field of organic waste management through open technologies.

1.1 Motivation

The management of organic waste represents one of the major current challenges in the environmental field, particularly in rural or hard-to-reach areas where adequate industrial infrastructure is lacking. Despite the growing interest in sustainable solutions such as composting, many of these practices lack control mechanisms that ensure their effectiveness, traceability, and replicability. This shortcoming hinders their large-scale adoption and limits their impact on environmental improvement.

At the same time, the maturity reached by certain digital technologies such as the low cost microcontrollers, lightweight communication protocols like MQTT, and open platforms such as FIWARE makes it possible to propose environmental monitoring solutions that are affordable, modular, and adaptable to diverse scenarios. These tools pave the way for systems capable of collecting and analyzing real-time data, improving decision-making, and facilitating the technical management of processes that previously relied on manual observation or empirical knowledge.

This project stems from the motivation to bring together these two realities: on one hand, the need to improve composting management in resource-constrained environments; and on the other, the potential of open, connected technologies to deliver viable, scalable, and replicable solutions. The opportunity to collaborate with Diputación de Badajoz and to leverage its existing FIWARE infrastructure also adds significant value to the project, as it facilitates integration with public services and deployment in real territorial initiatives.

In this context, the present Master's Thesis aims not only to demonstrate the technical feasibility of a monitoring system for smart composters, but also to serve as a first step toward more connected, sustainable, and accessible models of local environmental management.

2 Objectives

The main objective of this work is to create an intelligent and interconnected composting system using active devices that communicate via the MQTT and HTTP protocols. In addition, the system must be scalable and easily modifiable.

The proposed goals are:

- To develop a smart composting system.
- To interconnect and centralize the composting system using MQTT.
- To connect the composting systems with Diputación de Badajoz through the FIWARE platform.
- To build a robust infrastructure that ensures reliable communication, including error handling and fault management.
- Design scalable, easy to implement and modify system.
- To facilitate system deployment and modification.

3 State of the Art

3.1 Organic waste management and composting

3.1.1 Introduction

Management of urban organic waste represents one of the greatest environmental challenges in modern societies. Globally, it is estimated that organic waste accounts for between 40% and 60% of total municipal solid waste. These wastes mainly food scraps and garden waste, have high potential for valorization through biological processes such as composting.

Composting is an aerobic decomposition process of organic matter carried out by microorganisms, resulting in a stable, sanitized, and nutrient-rich product called compost. This product can be used as an organic soil amendment in agricultural or urban settings, thus closing the nutrient cycle and contributing to a circular economy.

There are multiple types of composting, among which household, community, and industrial composting stand out. Each of these approaches entails different levels of technical complexity, scalability, and requirements for controlling environmental parameters such as temperature, moisture, carbon-to-nitrogen (C/N) ratio, and aeration. Proper management of these parameters is essential to ensure the efficiency of the process and the quality of the final compost.

In this context, the integration of technological tools for the monitoring and automation of composting is gaining increasing relevance, particularly in urban, rural, and peri-urban environments where process efficiency and traceability are key.

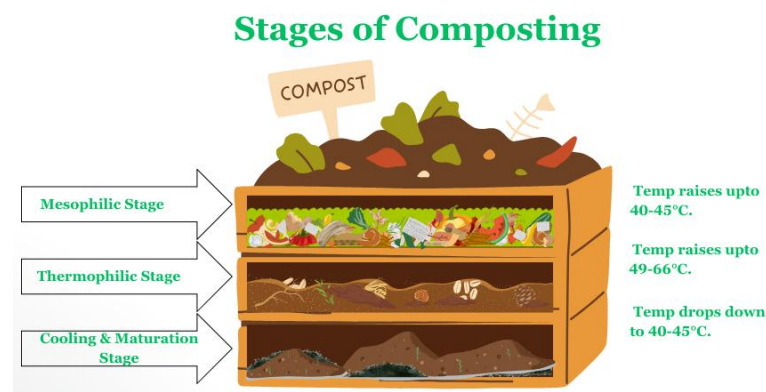


Ilustración 1: Composting stages [4]

3.1.2 Composting basics

The process involves breaking down organic matter into a humus-like material known as compost, which serves as an effective fertilizer for plants. Composting organisms require four key ingredients to function efficiently [5].

- Carbon is needed as an energy source; microbial oxidation of carbon generates the heat required for other parts of the composting process. Materials high in carbon tend to be brown and dry.
- Nitrogen is essential for the growth and reproduction of microorganisms that oxidize carbon. Nitrogen-rich materials are typically green and moist, and may also include colorful fruits and vegetables.
- Oxygen is required for the oxidation of carbon—the decomposition process itself. Aerobic bacteria need oxygen levels above 5% to carry out the necessary processes for composting.
- Water is needed in appropriate amounts to sustain microbial activity without creating localized anaerobic conditions.

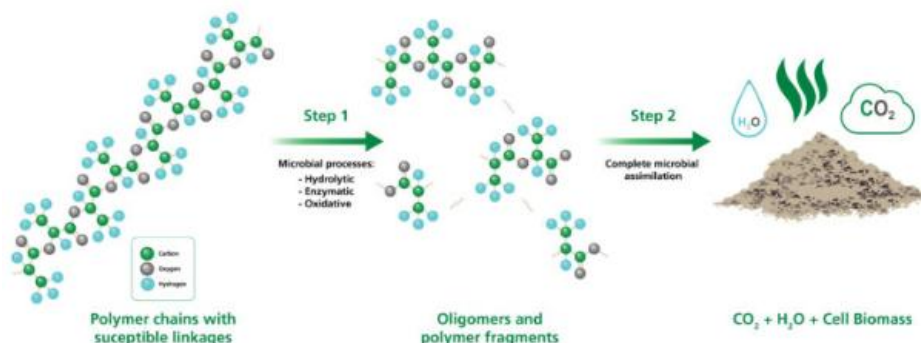


Ilustración 2: Organic stages of composting [6]

3.1.3 Different types of composting

Composting can be carried out at various scales and in different locations. The management of composting sites and facilities may be public, private, or a combination of both. Materials arrive at composting sites through a range of collection programs.

The fundamentals of composting remain the same regardless of the size or scale of the operation. However, the volume of composted materials, the types of feedstock, as well as the composting methods and equipment used, may vary. Composting operations that handle large volumes of material or specific types of feedstock, such as biosolids or

manure, may be subject to additional permitting or zoning requirements, depending on state and local regulations [7].

Home Composting

Home composting is carried out in residential settings, often in a backyard pile or a small enclosed bin. Vermicomposting, or worm composting accelerated (by the action of worms), is also an option for those practicing home composting. These systems primarily rely on food scraps (excluding meat, bones, and dairy products) and yard trimmings. By composting at home, the amount of waste sent to landfills or incinerators is reduced, and the resulting compost can be used in gardens, yards, or indoor plants [7].

Community Composting

Community composting is carried out at the neighborhood or community level and can be located at schools, universities, community gardens, farms, parks, religious institutions, and other spaces in urban, rural, and suburban areas. The defining feature of community composting is that both the process and the product (compost) remain as local as possible, while also engaging the community through participation and education. Compost generated from locally sourced materials is considered a community asset and is used within the same community where it is produced. The benefits of community composting include improved local soils, increased food security, local job creation, greener neighborhoods, and enhanced community engagement. Community composting contributes to the social, economic, and environmental well-being of the community [7].

On-site Composting

In situ composting is dedicated to composting materials generated at a specific site. A business, school, or other institution may compost materials it produces, such as food scraps from the cafeteria or garden pruning waste from the campus grounds. In situ composting reduces the costs associated with transporting organic materials offsite for processing or disposal, contributes to sustainability goals, and produces a valuable soil amendment that can be used in gardens and landscaping [7].

Municipal Composting

Municipal composting refers to programs run by local governments, sometimes known as curbside collection, in which yard waste or food scraps are collected from households and businesses or at designated drop-off points. These programs may be managed, funded, and operated at the municipal, city, county, or solid waste district level, or through a combination thereof, and can involve contracts with private companies or compost producers. Municipal composting programs can be integrated with other local waste management initiatives, such as recycling, biosolid management, and municipal solid waste disposal. A municipality may have its own composting capacity or transport collected materials to a large-scale centralized composting facility. The resulting compost may be made available to residents and used in municipal buildings, schools, parks, along roadways, and in green spaces [7].

Industrial Composting

Commercial and industrial composting takes place at large-scale composting facilities designed to handle high volumes of organic material. These facilities may provide composting services to residential, commercial, and institutional sectors. They can divert significant amounts of organic waste from disposal sites. Facilities may be centralized, sourcing raw materials from a broad regional area, or located at a specific industrial site serving that location. Commercial composting operations typically rely on tipping fees, contracts with local governments, and compost sales to fund their activities [7].



Ilustración 3: Industrial Composting [6]

3.1.4 Advantages of composting

Composting offers multiple advantages from environmental, agronomic, economic, and technological perspectives. Its proper implementation not only contributes to the efficient management of organic waste but also promotes sustainability and resource recovery across various contexts (domestic, community, industrial, and agricultural). The main benefits of composting are outlined below.

Reduction of organic load in landfills

From a logistical and environmental standpoint, composting allows for diverting a significant fraction of municipal solid waste (MSW) from the flow headed to landfills, where organic waste is responsible for generating leachates and greenhouse gases such as CH_4 and CO_2 [8].

Biological stabilisation and sanitisation

During the composting process, the organic matter undergoes a thermophilic phase (50–70 °C) in which pathogens, viable seeds, and phytotoxic compounds are eliminated [9].

Nutrient recovery and soil improvement

Compost is an organic amendment rich in nutrients such as nitrogen (N), phosphorus (P), and potassium (K), as well as essential micronutrients. Additionally, it provides stable organic matter (humus), which improves the soil's physical, chemical, and biological properties.

Reduction of fertiliser and chemical use

Compost can fully or partially replace the use of synthetic fertilizers and other inorganic amendments, especially in low-demand crops or organic farming [10].

Carbon sequestration and climate change mitigation

The application of compost to the soil contributes to long-term carbon sequestration in the form of stable organic matter, thereby reducing the amount of CO₂ in the atmosphere.

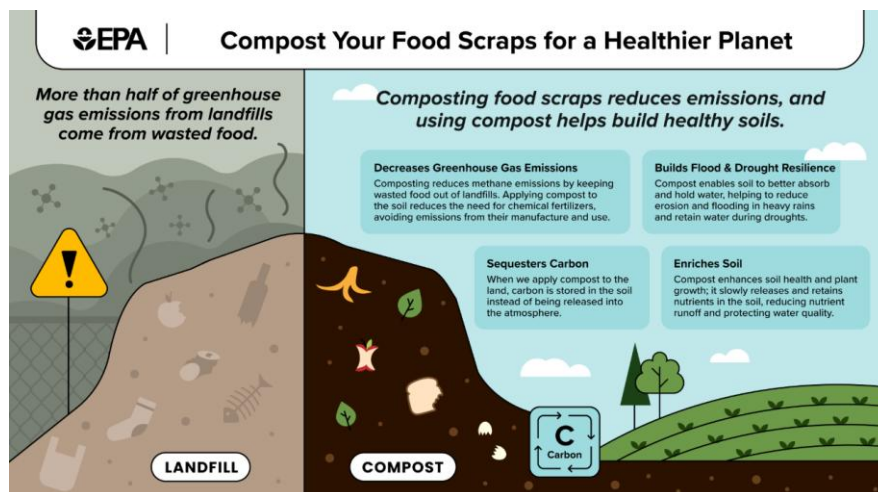


Ilustración 4: Advantages of composting [11]

3.2 IoT technologies in agriculture and environmental management

3.2.1 Introduction

The development of technologies based on the Internet of Things (IoT) has revolutionized the way agricultural and environmental settings are monitored and controlled. In agriculture, IoT has become a key tool within the framework of precision farming, enabling the continuous acquisition of data on soil conditions, crops, weather, irrigation levels, and pest presence.

Similarly, in the field of environmental management, IoT facilitates real-time data collection for monitoring air and water quality, waste levels, and critical parameters in recycling and waste treatment facilities.

IoT systems generally consist of distributed sensors, wireless communication platforms, processing units, and cloud services that allow the storage, analysis, make decisions, automate jobs, trigger alerts and visualization of collected data. Their implementation in composting systems enables continuous monitoring of parameters such as internal pile temperature, relative humidity, emitted gases (CO₂, CH₄), and leachate levels.

This monitoring allows for quicker decision-making, anomaly detection, prevention of human error, and optimization of process efficiency, which is particularly valuable in medium to large-scale composting systems or when strict quality standards need to be maintained.

3.2.2 Smart agriculture

As the demand for food increases, agriculture is rapidly adapting while simultaneously addressing climate change, water scarcity, and labor pressures. The use of IoT sensors in agriculture enhances efficiency, optimizes resource management, and promotes food security [12].

Agricultural IoT involves the use of sensors, connected equipment, software, and data science to continuously monitor and regulate farming processes. These devices collect data on soil, climate, crop health, animal behavior, and much more, which farmers can remotely monitor and utilize. Farmers can also automate operations, make informed decisions, and manage resources such as water, fertilizers, and pesticides through IoT technologies.

The global agricultural IoT market was valued at approximately USD 13.73 billion in 2023 and is projected to grow to USD 22.65 billion by 2028, with a Compound Annual Growth Rate (CAGR) of 10.7%. The rapid adoption of precision agriculture, livestock monitoring, and smart greenhouse solutions are key drivers of this growth [12].

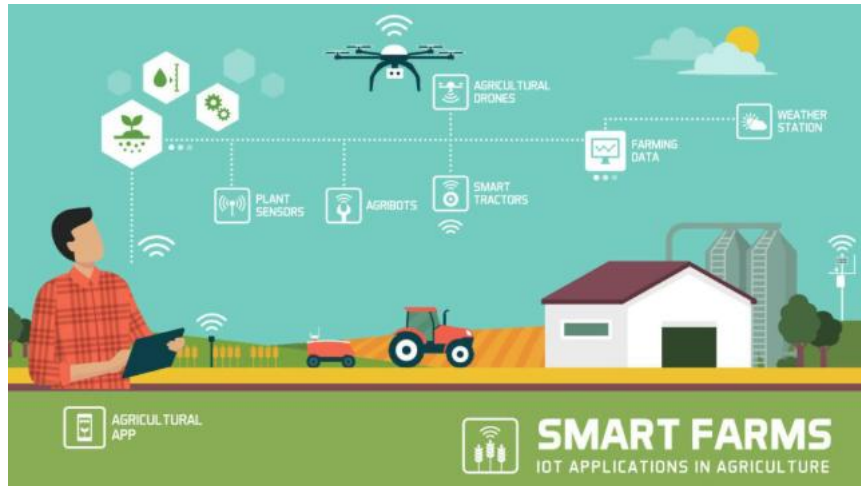


Ilustración 5: Smart agriculture [13]

3.3 IoT communication protocols

During this chapter, we will discuss the main MQTT communication protocols. This information can be extended by accessing the final thesis “Implementation of an interconnected workspace with active devices communicating using the MQTT protocol” of own authorship [14].

3.3.1 Introduction

In the Internet of Things, there is a vast number of devices that need to connect with each other. Furthermore, each of these devices performs different tasks, varying in data speed, size, and more. For this reason, we say that the IoT system is heterogeneous.

The interconnected devices in IoT need to communicate using lightweight protocols that do not require extensive CPU resource usage [14].

3.3.2 IoT communication architectures

Nowadays, there are two dominant architectures for data exchange protocols: Bus-Based and Broker-Based [14].

Broker-Based

In this type of architecture [15], also called Message Queue, the Broker controls the distribution of information. Among other things, it stores, forwards, filters, and prioritizes publish and subscribe requests. Devices can switch between publisher or subscriber roles depending on their objectives. Some of the most well-known protocols of this type are AMQP (Advanced Message Queuing Protocol), CoAP (Constrained Application Protocol), MQTT (Message Queue Telemetry Transport), and JMS (Java Message Service).

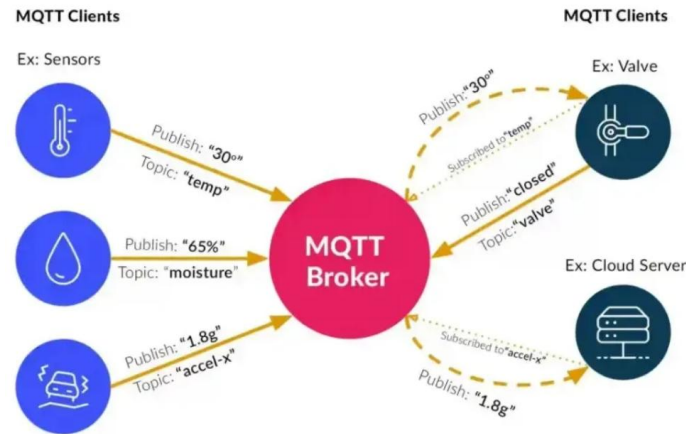


Ilustración 6: Broker-Based protocols (Ex: MQTT) [16]

Bus-Based

In this architecture [15], also known as Message Service, clients publish messages to a specific topic, which are delivered directly to the subscribers of that topic. There is no centralized broker. Some of the most well-known Bus-Based protocols include DDS (Data Distribution Service), REST (Representational State Transfer), and XMPP (Extensible Messaging and Presence Protocol).

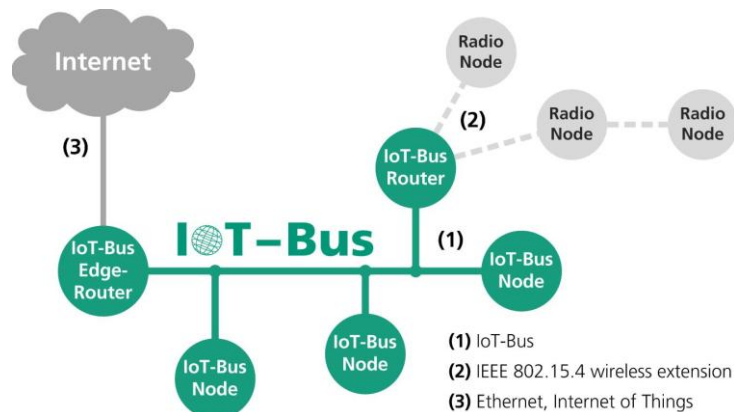


Ilustración 7: Bus-Based protocols [17]

3.3.3 Main IoT protocols

CoAP

CoAP (Constrained Applications Protocol) [18] is a client/server data transmission protocol similar to HTTP since it uses REST technology. A sensor typically acts as an information ‘server’ and the consumer as the ‘client’. It supports a one-to-one protocol for transmitting information between client and server.

CoAP employs content discovery, allowing devices to find each other for data exchange. It was designed for web interoperability, working with protocols like HTTP and RESTful as mentioned earlier, supporting asynchronous communications. This

protocol facilitates generating small packets and adapts to a data transfer model that observes state changes as they occur, rather than relying solely on events.

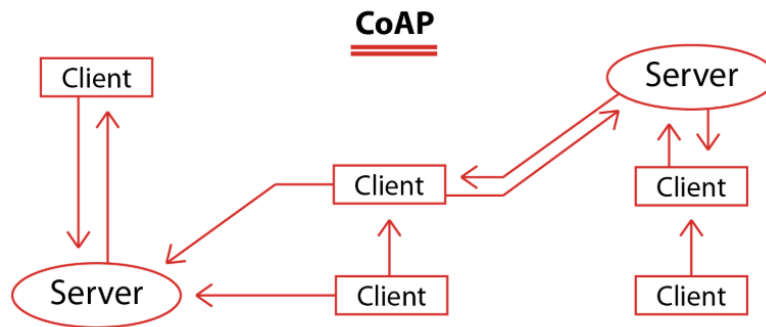


Ilustración 8: CoAP protocol [19]

AMQP

AMQP (Advanced Message Queuing Protocol) [20] is an application-layer protocol that originated in the financial sector with the purpose of replacing non-interoperable messaging systems. Its key features include message orientation, queues, routing (either point-to-point or publish/subscribe), reliability, and security, all managed through a centralized broker.

It provides message-oriented communication and controls flow with delivery guarantees, which can be at most once (where each message is delivered once or not at all), at least once (where each message is assured delivery but may be delivered multiple times), and exactly once (where each message is guaranteed to be delivered only once). It supports authentication and encryption based on SASL and TLS. AMQP is a reliable protocol due to its use of TCP (Transmission Control Protocol), which incorporates SSL and TLS.

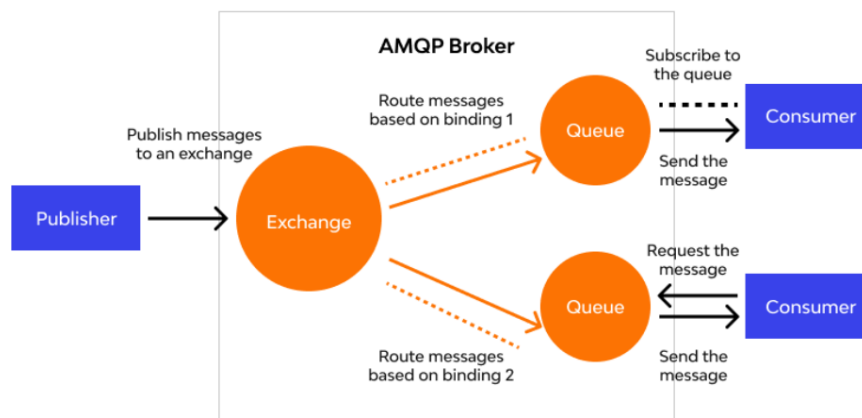


Ilustración 9: AMQP protocol [21]

MQTT

MQTT (Message Queuing Telemetry Transport) [22] is an open-source protocol for message exchange between multiple clients via a central intermediary. It was designed to be simple and easy to implement. Its architecture is based on a central broker and uses

a long-lasting TCP connection. MQTT supports a hierarchical file system structure. It is suitable for bidirectional communication over unreliable networks, where the cost per transmitted bit is high, and with low-power devices.

It is a lightweight protocol, well-suited for constrained environments. It typically uses asynchronous communication and can send automatic disconnection messages. There are different versions of MQTT designed to address specific limitations. By using TCP, it supports SSL/TLS security, although this may negatively impact network efficiency when the number of nodes exceeds one thousand, resulting in reduced performance. The protocol does not support automatic discovery of new clients.

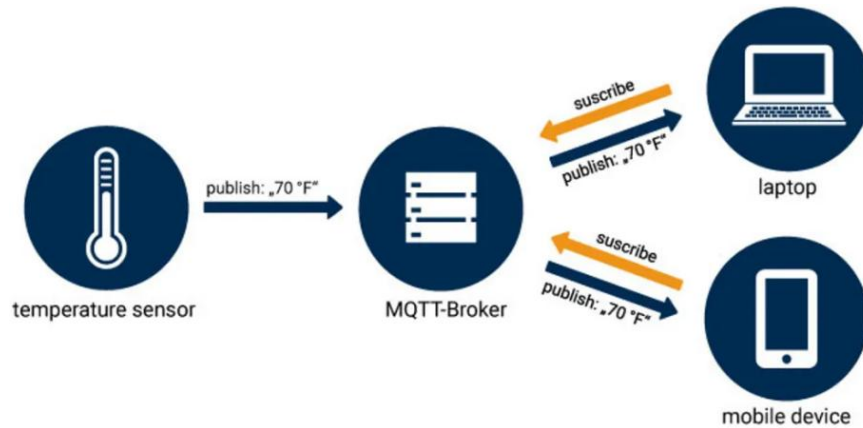


Ilustración 10: MQTT protocol [23]

JMS

JMS (Java Message Service) [24] is a middleware API designed to create, read, send, and receive messages between two or more clients. Based on Java Enterprise, it enables communication between different components in a loosely coupled, reliable, and asynchronous manner over TCP/IP.

This protocol supports both publish/subscribe and point-to-point messaging systems using message queues. Different Java classes can be used to communicate with various JMS providers through Java Naming and Directory services.

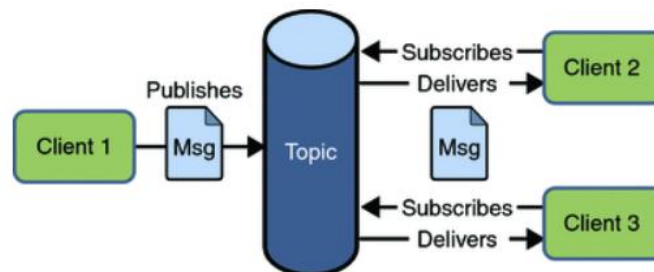


Ilustración 11: JMS protocol [25]

DDS

DDS (Data Distribution Service) [26] is a data language used to enable scalable, real-time, high-performance, and interoperable data exchange. It was designed for

financial trading, air traffic control, network management, and other critical big data applications.

DDS is a decentralized protocol with no intermediaries, enabling direct peer-to-peer communication between publishers and subscribers. It is language and OS-independent. DDS sends and receives data over UDP but can also operate over IP Multicast, TCP/IP, shared memory, and more. It supports real-time connectivity of a large number of devices with automatic discovery. DDS applications are decoupled and do not require user applications, which simplifies complex network programming. It includes QoS parameters to ensure reliable message delivery.

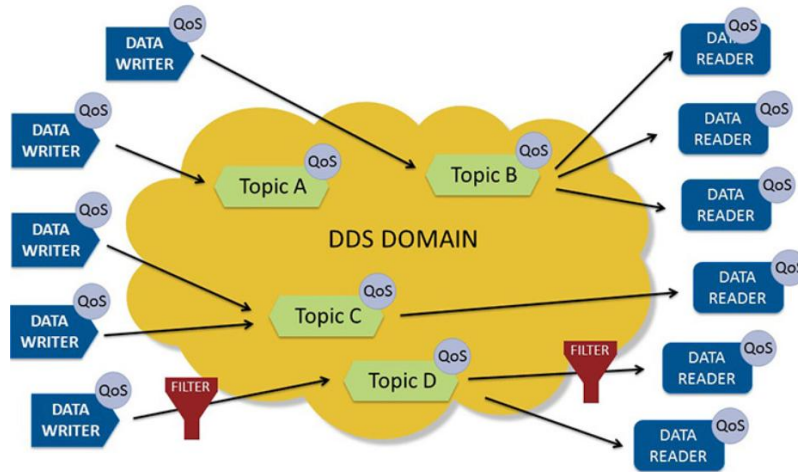


Ilustración 12: DDS protocol [27]

XMPP

XMPP (Extensible Messaging and Presence Protocol) [28] is a middleware protocol based on XML text messaging. It features a decentralized client-server model without intermediaries, and is used by text messaging applications.

It operates in near real-time and can scale massively to hundreds of thousands of nodes. Binary data must be encoded in base64 before transmission. XMPP is suitable for high-traffic, potentially complex devices that require enhanced security. It can isolate security without relying on TCP or web-based technologies.

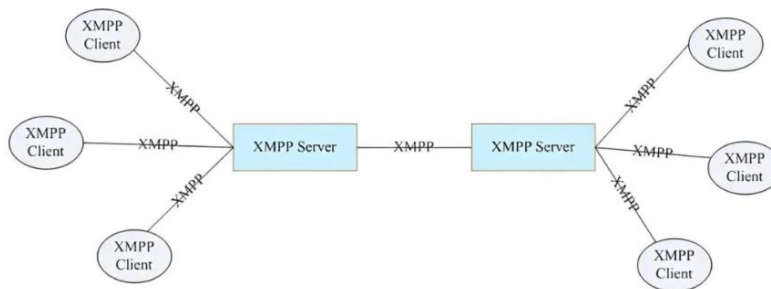


Ilustración 13: XMPP protocol [29]

3.3.4 MQTT communication structure

The MQTT protocol is built on top of TCP/IP, meaning both the client and the broker must rely on a TCP/IP stack [30].

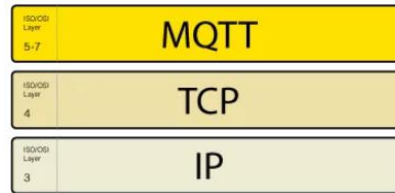


Ilustración 14: MQTT in the OSI model [30]

Connections in MQTT are always established between a client and a broker. Clients never connect directly to one another.

To initiate a connection, the client sends a CONNECT message to the broker, which responds with a CONNACK message and a status code. Once the connection is established, the broker keeps it open until the client explicitly sends a disconnection command or the connection is otherwise interrupted.

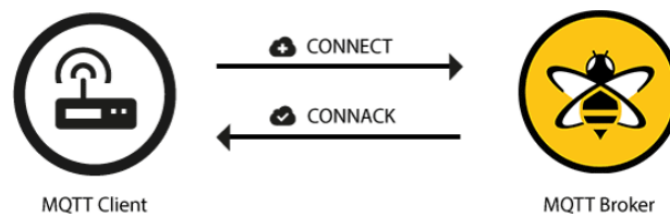


Ilustración 15: Initial connection broker-client MQTT [30]

Taking a closer look at the MQTT protocol, we can break down the structure of a typical message. As previously mentioned, MQTT messages are lightweight and follow a well-defined structure, consisting of the following parts.

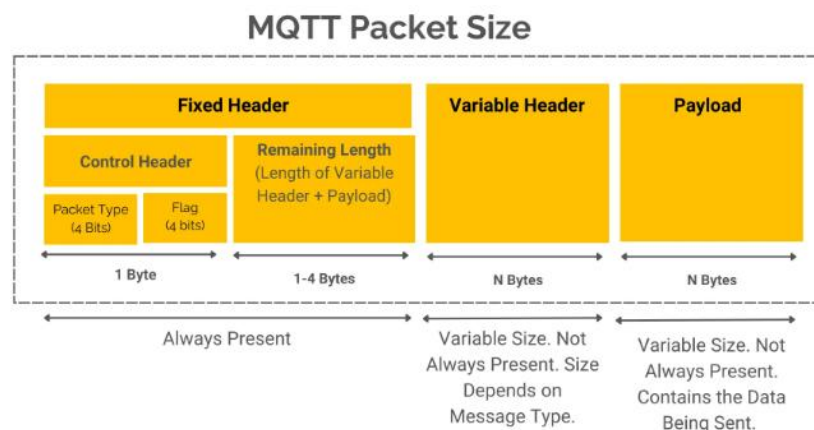


Ilustración 16: MQTT message structure [31]

The fixed header is responsible for identifying the type of MQTT control packet being sent, such as connect, disconnect, publish, subscribe, etc. It contains two key components.

- **Control field**
The first 4 bits of the fixed header specify the message type. Since 4 binary bits can represent 16 unique decimal values, MQTT can support up to 16 different message types [31].
- **Remaining length**
This is the second byte of the fixed header and indicates the total length of the remaining bytes in the packet. Essentially, it specifies the combined size of the variable header and the payload [31].

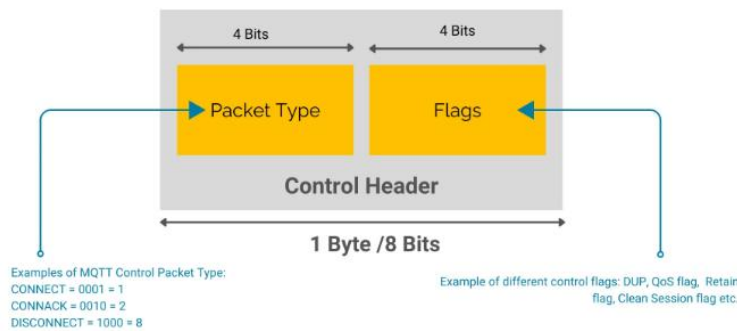


Ilustración 17: Fixed header structure in MQTT

The variable header section is not always required, especially for simple operations. However, it becomes important for more advanced use cases, as it contains additional packet-specific information.

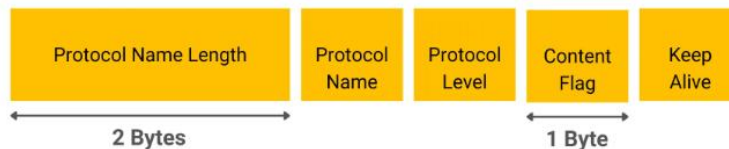


Ilustración 18: Variable header structure in MQTT

The payload is the actual content of the message. This is where the data being transmitted.

3.4 Existing smart composting solutions

3.4.1 Introduction

In recent years, smart composting has seen significant advancements thanks to the integration of many of the described technologies. This evolution responds to the need to improve the efficiency, traceability, and sustainability of organic waste treatment processes, both at domestic and industrial levels.

Current solutions enable real-time monitoring of critical parameters such as temperature, humidity, pH, and gases concentration, allowing for more precise control of the biological decomposition process.

Among the implemented technologies are sensors connected via wireless networks (Wi-Fi, LoRaWAN, NB-IoT), cloud platforms for data visualization and analysis, and automated systems that directly act on the process by regulating aeration, humidity, or material mixing. Additionally, some developments incorporate artificial intelligence or machine learning capabilities to optimize decision-making.

3.4.2 Composting in Extremadura

To continue, some examples of composting in Extremadura are described:

School composting - Compobox

The Compobox [32] is a composting machine invented by sisters Iris and Chanel Sopo and used at the school in El Torviscal. There, students separate organic waste and turn it into fertilizer. It's a pioneering initiative that other educational centers are already looking to replicate, and which has attracted interest from the company Promedio. It could potentially become the "fifth bin" in the recycling process

Parents, students, teachers, and other residents of El Torviscal are fully committed, united by a single goal: achieving zero waste. It's simple, fruit and vegetable scraps are taken to the school's Compobox container, and once turned into compost, they are returned to the garden cultivated on school grounds.

Awareness raising and training in Cáceres

In October 2023, the MásMedio Consortium [33], Cáceres Provincial Council launched the free course “Design, Management, and Treatment of Decentralized Composting Systems” (50 hours), aimed at, farmers, and companies, in line with EU regulations on organic waste collection.

In October 2024, the 13th “Composta en Red” [34] Seminar was held in Cáceres, featuring panels on home, community, agricultural, and school composting, as well as video composting and compost tasting sessions.

FORS plants and centralised composting

According to the Comprehensive Waste Plan of Extremadura (PIREX 2023–30), the region will deploy 12 composting plants for the organic fraction (FORS) by 2030 [35]. The first facility is set to begin construction soon in Llerena, with operations expected to start between late 2025 and early 2026. It will serve 31 municipalities [36].

3.4.3 Proposed solution

As we have seen, the implementation of reuse, generation, and treatment of organic matter, whether to produce compost or to minimize the environmental impact of excessive material consumption, is more relevant than ever.

Both Spain and, more specifically, Extremadura are committed to developing projects that support this goal. These range from small-scale initiatives in schools and DIY domestic composting, to large processing plants.

However, our system offers a middle ground, featuring large-scale deployment across the community while ensuring real self-sufficiency. Importantly, the required investment is not excessive and is expected to become cost-effective in a short time.

This proposed solution represents the midpoint between a full-scale processing plant and a domestic compost bin. When implemented in towns or cities, it can generate a significant amount of organic matter for reuse. Furthermore, as mentioned earlier, it does not require a large workforce or intensive physical maintenance, since all data will be available on the Diputación’s FIWARE platform, allowing the material to be managed when necessary.

4 Metodología

Para llevar a cabo el proyecto, hemos utilizado una metodología de desarrollo iterativa e incremental. En este tipo de desarrollos, el proyecto se planifica en diversos bloques temporales llamados iteraciones [37]

Las iteraciones pueden entenderse como pequeños proyectos en sí mismos: en cada una se repite un proceso de trabajo similar, con el objetivo de ofrecer un resultado completo sobre el producto final, permitiendo que el cliente obtenga beneficios de forma incremental. Para ello, cada requisito debe completarse dentro de una sola iteración, lo que implica llevar a cabo todas las tareas necesarias, incluidas las pruebas y la documentación, de modo que esté listo para ser entregado sin apenas esfuerzo adicional. Así, se evitan riesgos al final del proyecto relacionados con la entrega de funcionalidades.

En cada iteración, el equipo hace evolucionar el producto mediante entregas incrementales, basándose en lo ya desarrollado y añadiendo nuevos requisitos o mejorando los existentes. Un aspecto clave de este enfoque es la priorización de los objetivos según el valor que aportan al cliente [37].

Esta metodología ha sido elegida en conjunto con Espacio Fiware de Diputación de Badajoz, ya que a lo largo de todo el proyecto hemos mantenido un contacto continuo para el desarrollo de las fases del proyecto.

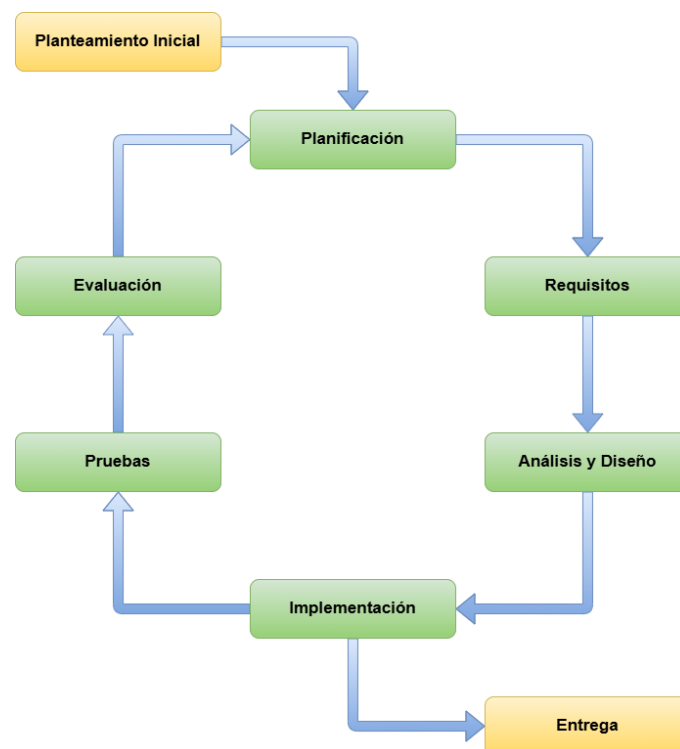


Ilustración 19: Esquema desarrollo iterativo

4.1 Fases del proyecto

4.1.1 Iteración 1

Para la primera iteración hemos definido los requisitos y objetivos principales del proyecto. Las necesidades con las que cuenta el proyecto y la forma más óptima para resolverlas.

Objetivos

- Análisis de requisitos y necesidades.
- Estudio de principales funcionalidades.

Resultados

- Necesidad de un sistema fácilmente desplegable y escalable. Que pueda de forma sencilla sensorizar uno o varios contenedores de compostaje y enviar los datos a Fiware.
- Sistema compuesto por sensores y captadores, servidor local y envío de datos a plataforma Fiware.

4.1.2 Iteración 2

Durante la segunda iteración, hemos estudiado las opciones tecnológicas disponibles para cumplir con los requisitos del sistema. Se han fijado las tecnologías principales de las que se compondrá el proyecto.

Objetivos

- Análisis de tecnologías IoT (dispositivos y comunicación).
- Análisis de tecnologías de infraestructura y servidores.

Resultados

- Elección de sistemas de sensorización y captación de datos del compost (sensores y M5Stack Tough basado en ESP32 [38]). Además, se utilizará el protocolo MQTT para la comunicación de los dispositivos con el servidor local.
- Centralización de captadores e información en un servidor local. Sistema en forma de estrella para desde ahí enviar la información a Fiware. Servidor local desplegado en Raspberry con sistema operativo Debian GNU Linux y Docker.

4.1.3 Iteración 3

Una vez definidos los requisitos y las tecnologías, hemos creado un esqueleto tecnológico del proyecto. Para ello se ha implementado la primera infraestructura a modo *MVP (Minimum viable product / Producto mínimo viable)*.

Objetivos

- Despliegue de infraestructura de servidor local básica.
- Programación de M5Stack Tough y comunicación MQTT.
- Inicio de documentación.

Resultados

- Utilizando Docker, se han levantado los contenedores de Mosquitto (MQTT bróker) y Node-Red (encargado de lógica y captación local de datos).
- Creación de librerías para trabajar con M5Stack Tough y comenzar a enviar y recibir datos mediante MQTT.
- Se comienza a documentar el desarrollo del proyecto.

4.1.4 Iteración 4

En esta iteración, nos hemos centrado en comprobar el funcionamiento del sistema. Validando diferentes aspectos esenciales de la captación de datos, transmisión y recopilación.

Objetivos

- Pruebas de comunicación.
- Pruebas de infraestructura.

Resultados

- Confirmación de correcto funcionamiento del sistema, tanto captación como transmisión mediante MQTT y REST.
- Correcto funcionamiento de infraestructura y facilidad de despliegue.

4.1.5 Iteracion 5

Tras comprobar que la parte software del sistema es funcional, hemos comenzado con la parte de hardware, conectando sensores y codificando los datos.

Objetivos

- Conexión de sensores a M5Stack Tough.
- Programación de librerías para lectura de sensores y codificación de datos.
- Pruebas de integración M5Stack Tough – Sensores.

Resultados

- Montaje de sistema físico, con sensores conectados a M5Stack Tough, tomando medidas y en funcionamiento.
- Pruebas de captación con entorno controlado, no de compostaje real.

4.1.6 Iteración 6

En esta iteración, ya contamos con un sistema físico montado y una infraestructura desplegada y estable. Por ello, una vez terminada la parte de desarrollo, hemos pasado a la de control de calidad.

Objetivos

- Asegurar la calidad del sistema.
- Finalizar la documentación y entregar el proyecto.

Resultados

- Sistema en funcionamiento y ajustado tras revisión de las pruebas.
- Entrega del proyecto planificada y fijada.

4.2 Planificación del proyecto

Para poder tener una referencia visual de las fases del proyecto, se ha creado un diagrama de Gantt. De esta forma, podemos delimitar la línea temporal de cada parte.

Objetivos	Enero				Febrero				Marzo				Abril				Mayo				Junio			
	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4
Análisis requisitos y necesidades																								
Estudio principales funcionalidades																								
Análisis tecnologías IoT																								
Análisis tecnologías infra																								
Despliegue infra MVP																								
Programación M5 MVP																								
Documentación																								
Pruebas comunicación																								
Pruebas infraestructura																								
Conexión Sensores-M5																								
Programación librerías sensores																								
Pruebas integración M5-Sensores																								
Pruebas de calidad																								
Entrega del proyecto																								

5 Desarrollo e implementación

En este capítulo, se describirá de forma detallada todo el desarrollo e implementación llevado a cabo en el proyecto, las diferentes arquitecturas (hardware y software), los sistemas de comunicación y el despliegue físico.

4.3 Arquitectura software

4.3.1 Diseño general del sistema

Como hemos mencionado anteriormente, el sistema software ha sido diseñado con una arquitectura modular, orientada a microservicios, permitiendo escalabilidad, mantenibilidad y una integración eficiente de sus componentes.

Contamos con un control de versiones [39] para los códigos utilizados en el desarrollo del proyecto. Utilizando la plataforma GitHub, se puede acceder a todos los ficheros utilizados entrando en el siguiente enlace.

TFM-IDG-SmartComposters

<https://github.com/idiestro/TFM-IDG-SmartComposters.git>

TFM-IDG-SmartComposters

Public

Pin

Unwatch

main

1 Branch

0 Tags

Go to file

Add file

Code

idiestro

Update Node-Red - Flows.json to fix Fiware conection

8e48410 · 10 hours ago

53 Commits

	.vscode	Update Settings - vscode	2 months ago
	docker	Update Node-Red - Flows.json to fix Fiware conection	10 hours ago
	include	Update Main - Speed port	last week
	lib	Update MeasuresDataHelper - Use extern to declare Measur...	yesterday
	src	Update Main - Speed port	last week
	.gitignore	Update gitignore	last month
	README.md	Update README - add docker and deploy system	2 weeks ago
	platformio.ini	Update Measures Reader and helper	last month

Ilustración 20: Repositorio GitHub TFM-IDG-SmartComposters

El sistema se compone de tres partes principales, las cuales componen los pilares de la arquitectura Software: sensorización y adquisición de datos, comunicación y procesamiento y backend y gestión.

Sensorización y adquisición de datos

Como dispositivo embebido hemos utilizado el M5Stack Tough. Este se encarga de la captación de datos, su procesamiento básico y el envío al servidor local.

Comunicación y procesamiento

Para la comunicación entre el M5 Stack Tough y el servidor local (Raspberry Pi 5) se utiliza el protocolo MQTT. Para poder enviar la información, esta debe ser tratada

ligeramente por el sistema embebido; para aligerar la carga y optimizar su manipulación en posteriores etapas.

Backend y gestión

Contamos con un servidor local, el cual recibe los datos de los sistemas de compostaje. En este paso, hacemos un tratado ligero de datos para adaptarlo a cadenas posteriores y aplicamos sistemas de control y alertas en la parte de borde. Los datos procesados en local serán enviados a Fiware para tratarlos posteriormente en profundidad.

Visualización y control

Aunque el sistema ha sido diseñado para contar con estas características, actualmente no se ha implementado en este proyecto. El desarrollo de esta parte frontend saldrá a licitación para que empresas privadas junto con Diputación de Badajoz puedan encargarse de ampliar esta parte.

Actualmente, solo contamos con visualización de datos en la parte del servidor local, utilizando Node-Red y sus características para crear tableros y gráficos.

4.3.2 Tecnologías y lenguajes utilizados

A continuación, describimos las principales tecnologías y lenguajes utilizados para el desarrollo del sistema.

Lenguajes de programación

Para llevar a cabo el sistema, se han utilizado diversos lenguajes de programación. De esta forma, hemos podido abarcar todas las características necesarias. Estos se extienden a través de las distintas capas que componen el proyecto.

C++

Es un lenguaje multiplataforma, el cual se puede utilizar para crear aplicaciones de alto rendimiento. Es una extensión del lenguaje C y ofrece a los programadores un alto nivel de control sobre los recursos del sistema y la memoria [40].

Esto, nos es de gran ayuda para programar nuestro M5, ya que, aunque las instrucciones por si solas no cuentan con gran complejidad, son muchas. Así, podemos dividir las en funciones, agruparlas y encapsularlas en clases y métodos para una mejor jerarquización y posterior ampliación de las funcionalidades del sistema.

Javascript

Es un lenguaje de programación ligero, interpretado, o compilado justo-a-tiempo (just-in-time) con funciones de primera clase. JavaScript es un lenguaje de programación basado en prototipos, multiparadigma, de un solo hilo, dinámico, con soporte para programación orientada a objetos, imperativa y declarativa [41].

En este proyecto, lo hemos utilizado para crear funciones personalizadas dentro de Node-Red. Al ser un lenguaje ligero y orientado a programación funcional, nos permite

rápidamente crear funciones complejas, sin tener que estructurar clases ni módulos auxiliares.

Bash

Bash es un programa que interpreta comandos, normalmente utilizado a través de una consola donde el usuario introduce instrucciones. Además, permite ejecutar instrucciones almacenadas en archivos, conocidos como scripts, los cuales están escritos en su propio lenguaje. Estos archivos sirven para automatizar procesos y agrupar distintos comandos y estructuras de control en un mismo lugar. [42].

Para este proyecto, Bash se ha utilizado en la automatización del despliegue de la infraestructura backend (Docker, Mosquitto, Node-Red). De esta forma, facilitamos la instalación de tecnologías y dependencias a la hora de crear un nuevo sistema.

Protocolos de comunicación

Para la comunicación entre las distintas partes del sistema hemos utilizado principalmente dos protocolos: MQTT y HTTP

MQTT

MQTT [43] es un protocolo de mensajería estándar OASIS [44] para el Internet de las Cosas (IoT). Está diseñado como un transporte de mensajería de publicación/suscripción extremadamente ligero que resulta ideal para conectar dispositivos remotos con una huella de código pequeña y un ancho de banda de red mínimo.

Nos es especialmente útil para la comunicación entre el M5 Stack Tough y el servidor local. Siendo un protocolo ligero y fácilmente implementable en estos dispositivos.

HTTP

HTTP [45], de sus siglas en inglés: "*Hypertext Transfer Protocol*", es el nombre de un protocolo el cual nos permite realizar una petición de datos y recursos, como pueden ser documentos HTML. Es la base de cualquier intercambio de datos en la Web, y un protocolo de estructura cliente-servidor, esto quiere decir que una petición de datos es iniciada por el elemento que recibirá los datos (el cliente).

Este protocolo nos permite enviar información a Fiware de forma segura y asegurando una calidad en la transmisión de información, gracias a sus características.

Frameworks y plataformas

En esta sección, explicaremos brevemente los frameworkd y plataformas utilizados para el desarrollo del proyecto.

Fiware

Es una plataforma, impulsada por la Unión Europea [46], para el desarrollo y despliegue global de aplicaciones de Internet del Futuro. Fiware intenta proveer de una arquitectura totalmente abierta, pública y libre, así como de un conjunto de especificaciones que permita a los desarrolladores, proveedores de servicios, empresas y

otras organizaciones desarrollar productos que satisfagan sus necesidades, sin dejar de ser abierta e innovadora.

Fiware ha sido adoptada por la iniciativa de la Unión Europea, *Connecting Europe Facilities* [47] como el componente clave para la gestión de datos (*Context Broker*). Las aplicaciones de este componente lideran las aplicaciones de *Open Data*, *Data Lakes* y gestores de contexto.

Para este proyecto, Fiware se encargará de recibir la información del sistema para, posteriormente, poder trabajar con los datos, analizarlos y sacar conclusiones para sistemas de control del compostaje.

Node-Red

Es una herramienta de programación basada en flujo de datos [48], desarrollada originalmente por el equipo de *IBM Emerging Technology Services* y ahora parte de *OpenJS Foundation* [49].

Node-Red [50] ofrece un editor de flujos basado en navegador, que facilita la interconexión de flujos utilizando la amplia gama de nodos de la paleta. Los flujos pueden implementarse en el entorno de ejecución con un solo clic. Se pueden crear funciones de JavaScript dentro del editor utilizando un editor de texto enriquecido. Una biblioteca incorporada permite guardar funciones, plantillas o flujos útiles para reutilizarlos.

Se encargará de gestionar toda la lógica, transformación y tratamiento de los datos en el servidor local. También enviará los datos procesados a Fiware.

Docker

Docker [51] es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos. Utiliza características de aislamiento de recursos del kernel Linux, tales como cgroups y espacios de nombres (namespaces) para permitir que "contenedores" independientes se ejecuten dentro de una sola instancia de Linux, evitando la sobrecarga de iniciar y mantener máquinas virtuales.

En este sistema, Docker nos permite dividir la aplicación en microservicios, independizando los contenedores de cada módulo y añadiendo facilidad de despliegue, mantenimiento y escalabilidad.

Docker Compose

Docker Compose [52], es una herramienta para definir y ejecutar aplicaciones multicontenedor. Es la clave para lograr una experiencia de desarrollo e implementación optimizada y eficiente. Compose simplifica el control de toda la pila de aplicaciones, facilitando la gestión de servicios, redes y volúmenes en un único archivo de configuración YAML. Con un solo comando, se crean e inician todos los servicios desde el archivo de configuración.

Docker Compose nos permite prefijar la configuración de la infraestructura utilizando contenedores, conectarla y configurarla automáticamente. Todo gracias a su fichero de configuración *docker-compose.yml*.

Platformio

PlatformIO [53] aplica la última tecnología de software escalable y flexible al mercado de sistemas embebidos, un área tradicionalmente atendida por herramientas de software complejas que ingenieros de hardware experimentados han aprendido con el tiempo (a menudo con dificultad). En cambio, con PlatformIO, los usuarios pueden ser aficionados o profesionales. Pueden importar el clásico sketch "Blink" de Arduino o desarrollar un sofisticado programa embebido en C de bajo nivel para un producto comercial. El código de ejemplo para cualquier framework compatible se puede compilar y subir a una plataforma de destino en minutos.

Esta herramienta, ha supuesto un avance significativo en la facilidad de programación del sistema embebido (M5Stack Tough). Permitiendo la programación del dispositivo desde IDE's potentes como VSCode [54] y su facilidad de compilación, pruebas y despliegue en el dispositivo físico.

Broker MQTT

Aunque existen gran variedad de brokers MQTT, para el proyecto hemos utilizado Mosquitto [55]. Es un bróker sencillo, sin embargo, cuenta con funcionalidades avanzadas que nos permiten prototipar el sistema con gran precisión y exigencia.

Eclipse Mosquitto [55] es un agente de mensajes de código abierto (con licencia EPL/EDL) que implementa las versiones 5.0, 3.1.1 y 3.1 del protocolo MQTT. Mosquitto es ligero y compatible con todo tipo de dispositivos, desde ordenadores de placa única de bajo consumo hasta servidores completos. El proyecto Mosquitto también proporciona una biblioteca C para implementar clientes MQTT y los muy populares clientes de línea de comandos MQTT `mosquitto_pub` y `mosquitto_sub`.

Mosquitto es parte de la Fundación Eclipse, y es un proyecto de *iot.eclipse.org*, con su desarrollo impulsado por Cedalo [56].

4.3.3 Estructura de módulos y servicios

A lo largo de este apartado, explicaremos más detalladamente la programación y funcionalidades de cada módulo software que compone el sistema. Podemos dividirlos en tres grandes grupos: sistema embebido, *backend* y comunicación.

Sistema embebido

Como sistema embebido, contamos con el M5Stack Tough, que utiliza el microcontrolador ESP32 [38]. A continuación, se detalla la estructura de programación que se ha utilizado para la configuración del dispositivo.

Para la toma de medidas, encontramos la librería '*MeasuresReader*'. Se encarga de configurar los sensores físicos y obtener sus valores de medición. Se puede ver el código de esta librería en los anexos.

MeasuresReader.h: es la interfaz pública donde definimos las clases y las funciones para la captación de información de los sensores. Puede verse el código en profundidad en el Anexo 1. Estas funciones son:

- `MeasuresReader`: constructor de clase.
- `initSensors`: llamada a la inicialización de los sensores.
- `getEnvironmentTemperature`: obtención de la temperatura del entorno.
- `getEnvironmentHumidity`: obtención de la humedad del entorno.
- `getEnvironmentGasO2`: obtención del nivel de O2 del entorno.
- `getEnvironmentGasCO2`: obtención del nivel de CO2 del entorno.
- `getBiomassTemperature`: obtención de la temperatura del compost.
- `getBiomassWeight`: obtención del peso total del compost.
- `getBiomassFluidAmount`: obtención de la cantidad de lixiviados del compost.
- `getBiomassPh`: obtención del Ph del compost.
- `initI2CWire`: funciones de iniciación para I2C.
- `initEnvIv`: funciones de iniciación del sensor ENV IV.
- `initEns160`: funciones de iniciación del sensor ENS160.
- `initDS18B20`: funciones de iniciación del sensor DS18B20.

MeasuresReader.cpp: implementación del código fuente que define el comportamiento de las funciones declaradas en la cabecera, para la lectura de sensores (Anexo 2).

En la sección de comunicación, hemos combinado dos funcionalidades, la conexión Wifi y las transmisiones MQTT. Aquí encontramos la librería '*MqttConnection*', encargada de toda la comunicación del sistema embebido con el bróker.

MqttConnection.h: es la interfaz pública donde definimos las clases y las funciones para la conexión Wifi y la comunicación MQTT. Puede verse el código en profundidad en el Anexo 3. Estas funciones son:

- `wifiConnection`: función de gestión de la conexión Wifi y la reconexión en caso necesario.
- `brokerConnection`: función encargada de la conexión con el bróker Mqtt y la reconexión en caso necesario.
- `MqttConnection`: constructor de clase, inicializa el cliente ESP y configura el Id de cliente.
- `init`: función de inicio del sistema, comenzando por la conexión Wifi.
- `reconnect`: función conjunta de reconexión con Wifi, bróker Mqtt y suscripción al tópico por defecto.
- `topicSubscription`: función de suscripción a nuevo tópico.
- `topicPublication`: función de publicación de mensaje en tópico.

- `setClientId`: configurar el Id de cliente para el dispositivo Mqtt.
- `loop`: función para mantener conexión activa Mqtt y recibir mensajes.
- `handleCallback`: función Callback para la recepción de mensajes Mqtt.
- `getMqttClient`: función para obtener el clienteMqtt.
- `getClientId`: obtención de Id de cliente.

MqttConnection.cpp: implementación del código fuente que define el comportamiento de las funciones declaradas en la cabecera, para conexiones Mqtt (Anexo 4).

También se han definido varias funciones de ayuda y auxiliares de cara a la captación, manipulación y transmisión de la información por parte del dispositivo. Se compone de las librerías '*MeasuresDataHelper*', '*LogsUtils*', '*JsonUtils*' y '*ConfigReader*'.

Estas librerías se explicarán de forma más ligera que las anteriores. Puede verse más documentación en el fichero *README.md* del repositorio de código.

- *MeasuresDataHelper*: librería encargada de orquestar la lectura de sensores y la creación del mensaje en formato Json que se enviará al bróker Mqtt. Puede verse el código en los Anexos 5 y 6.
- *LogsUtils*: librería con funciones de utilidad para mostrar los logs del sistema tanto por consola de comandos como por la pantalla del M5Stack. Puede verse el código en los Anexos 7 y 8.
- *JsonUtils*: esta librería, se encarga de trabajar con los ficheros y variables de formato Json. Cargando un fichero por defecto y permitiendo su modificación. Añadiendo los valores de las mediciones previamente obtenidos. Puede verse el código en los Anexos 9 y 10.
- *ConfigReader*: encargada de leer datos de configuración y decodificarlos. Esto se debe a que los datos sensibles se encuentran codificados dentro del código para evitar filtraciones a personas no autorizadas. Puede verse el código en los Anexos 11 y 12.

Continuando con las librerías de ayuda, también se han implementado las variables de configuración generales del sistema embebido. Estos datos están insertados en cabeceras, dentro de la carpeta '*include*'. Aquí encontramos tres ficheros importantes:

- *mainConfig*: cuenta con la configuración general del sistema, como por ejemplo la velocidad del puerto serie.
- *mqttConnectionConfig*: almacena las variables de las conexiones MQTT. Servidor, puerto y tópicos.
- *wifiConfig*: aquí encontramos la configuración para conectarse a la red Wifi. En este caso, la contraseña de la red está encriptada en Base64 para no exponerla de forma pública en el código.

Almacenar estas variables en cabeceras, nos permite poder hacer cambios rápidos en la configuración completa del proyecto sin necesidad de profundizar en el código.

Para final con la programación del controlador, tenemos el fichero principal que cargaremos en el M5Stack Tough, 'main.cpp'. Aquí, como es habitual encontramos dos funciones, 'loop' y 'setup'. La función 'setup' se encarga de la configuración inicial del sistema. Mientras que la función 'loop' es el bucle que se repetirá de forma ininterrumpida hasta que apaguemos el sistema.

```
25 void setup(void) {
26     //Init M5 and serial port
27     auto cfg = M5.config();
28     M5.begin(cfg);
29     Serial.begin(PORT_SPEED);
30     //Initial message
31     LogsUtils::printLog("---Iniciando sistema---");
32     //Init wifi and broker connection
33     mqttConnection.init();
34     //Init measures reader
35     measuresReader.initSensors();
36     LogsUtils::printLog("---Sistema iniciado---");
37     //Wait after setup
38     delay(5000);
39 }
```

Ilustración 21: Función 'setup' del fichero main.cpp

```
41 void loop(void) {
42     // Initial CPU pause
43     delay(10);
44
45     //Mqtt Connection loop
46     mqttConnection.loop();
47
48     // Read values and send messages
49     static unsigned long lastPublishTime = 0;
50     unsigned long currentMillis = millis();
51
52     if (currentMillis - lastPublishTime >= 10000) { // 10 seconds
53         lastPublishTime = currentMillis;
54         //Get measures from sensors and set into final payload
55         String finalPayload = measuresDataHelper.setMeasuresIntoJson();
56         //Publish measures to MQTT broker
57         mqttConnection.topicPublication(finalPayload);
58     }
59 }
```

Ilustración 22: Función 'loop' del fichero main.cpp

Backend

Definimos backend como la parte del sistema que se encuentra del lado del servidor, que implementará las lógicas y las funciones de traspaso y tratamiento de la información. Podemos dividirlo en dos secciones: servidor local y Fiware.

Para implementar el servidor local, hemos utilizado una Raspberry Pi 5. Con un sistema operativo Linux, concretamente con la distribución Debian GNU, versión 12.

Siguiendo con la arquitectura de microservicios anteriormente mencionada, hemos utilizado Docker y Docker Compose para desplegar los contenedores que contendrán las imágenes de Node-Red (versión 4.0.9) y Mosquitto (versión 2.0.21).

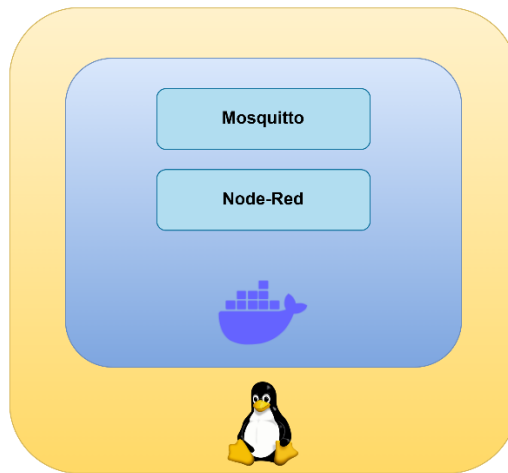


Ilustración 23: Sistema Linux y Docker en servidor local

Para poder facilitar el despliegue de la infraestructura en el servidor local, nos hemos ayudado de Docker Compose junto con Bash. Puede verse un ejemplo en la siguiente ilustración.

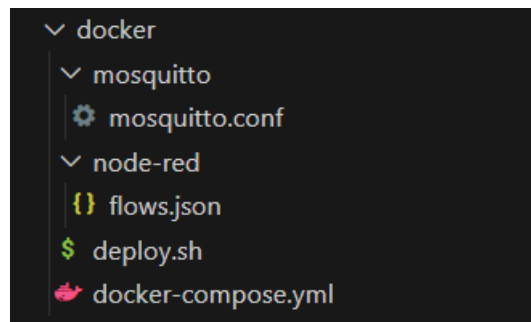


Ilustración 24: Ficheros de despliegue en servidor local

Utilizar Docker Compose permite definir y ejecutar aplicaciones multicontenedor con Docker. Usamos un archivo `docker-compose.yml` para configurar los servicios, redes y volúmenes necesarios para una aplicación. Luego, se puede levantar todo el entorno con un solo comando.

Nuestro '`docker-compose.yml`' (más información en el Anexo 13) lanza dos servicios: el broker MQTT Mosquitto y el entorno de virtualización Node-Red. Ambos están conectados a una red virtual común llamada '`iot-net`' para comunicarse entre sí.

Un fichero ejecutable de Bash '`setup.sh`' nos permite automatizar el despliegue del stack con Docker. De esta forma, nos configura de forma automatizada el sistema. El proceso de este ejecutable se divide en:

- Actualización del repositorio: utilizando '`git pull`' actualiza el repositorio local por si se han hecho cambios.
- Instalación de Docker: comprueba si Docker se encuentra instalado en el sistema. De no ser así, lo instala.

- Instalación de Docker Compose: comprueba si Docker Compose se encuentra instalado en el sistema. De no ser así, lo instala
- Creación de carpetas de persistencia: crea las carpetas necesarias para la persistencia de datos del sistema.
- Carga de ficheros de configuración: carga los ficheros de configuración para Mosquitto y Node-Red
- Despliegue de contenedores: ejecuta los contenedores y los mantiene corriendo en la máquina

Mosquitto hace la función de broker MQTT. Aunque su configuración puede extenderse bastante gracias a sus muchas posibilidades, nuestra configuración inicial ha sido mínima. centrándonos en: permitir conexiones anónimas y utilizar el puerto 1884. Podrían añadirse configuraciones adicionales de seguridad, persistencia, etc. Pero esto se plantea como ampliación futura.

Nuestro fichero de configuración cuenta con:

```
allow_anonymous true
listener 1884
```

Toda la configuración necesaria para Node-Red la encontramos en el fichero ‘flows.json’. Aquí, podemos encontrar un total de cinco flujos: Simulador de Datos, Conexiones MQTT, UI Control/Dashboards, Computación de Borde y Conexión Fiware

Desarrollados e imprescindibles para el funcionamiento del sistema son tres, los cuales detallamos a continuación.

Conexiones MQTT

Es el flujo encargado de gestionar las conexiones MQTT y recibir la información de los dispositivos físicos.

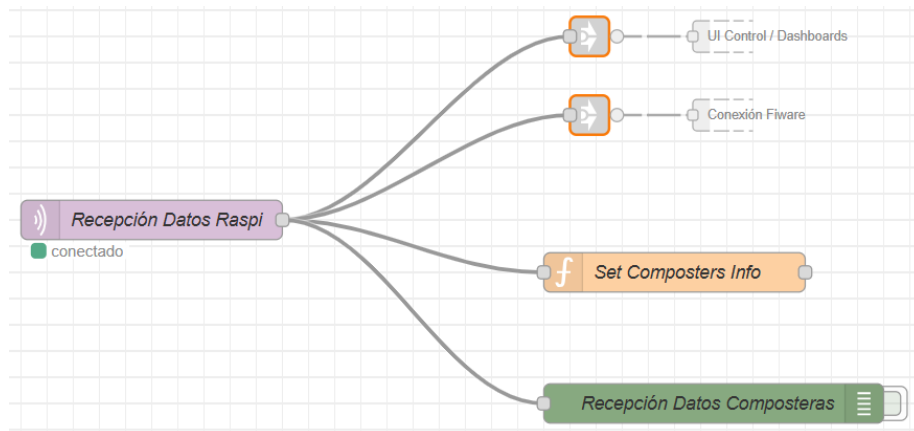


Ilustración 25: Node-Red, flujo Conexiones MQTT

En la ilustración anterior podemos ver los nodos del flujo mencionado, los cuales se explican a continuación:

- *Recepción Datos Raspi*: conexión con bróker MQTT para recibir datos del M5 Stack Tough.
- *Set Composters info*: código Javascript para extraer únicamente la información del sistema de compostaje y no las mediciones. Más información en el anexo Anexo 14.
- *Módulo de depuración*: para poder depurar las distintas etapas del sistema.
- *Envío a otros flujos*: envío de la información al flujo de 'UI Control/Dashboards' y 'Fiware'.

UI Control/Dashboards

Es el flujo encargado de extraer los datos del mensaje MQTT y representarlo en gráficas atractivas para el usuario.

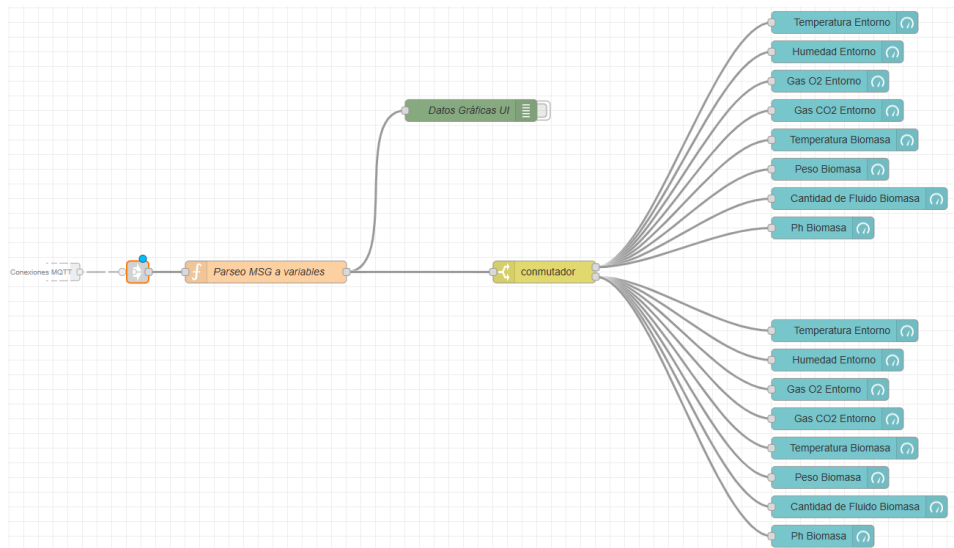


Ilustración 26: Node-Red, flujo UI Control/Dashboards

En la ilustración anterior podemos ver los nodos del flujo mencionado, los cuales se explican a continuación:

- *Parseo MSG a variables*: código Javascript para extraer el payload del mensaje.
- *Módulo de depuración*: para poder depurar las distintas etapas del sistema.
- *Conmutador*: selector de gráfica en función del id de la compostera que envía la información.
- *Gráficas*: representación gráfica en una interfaz de usuario de las medias obtenidas:



Ilustración 27: Node Red, gráficas de datos

Conexión Fiware

Es el flujo encargado de enviar los datos a Fiware. Solicita el token de autorización si es necesario y envía la información obtenida de las composteras.

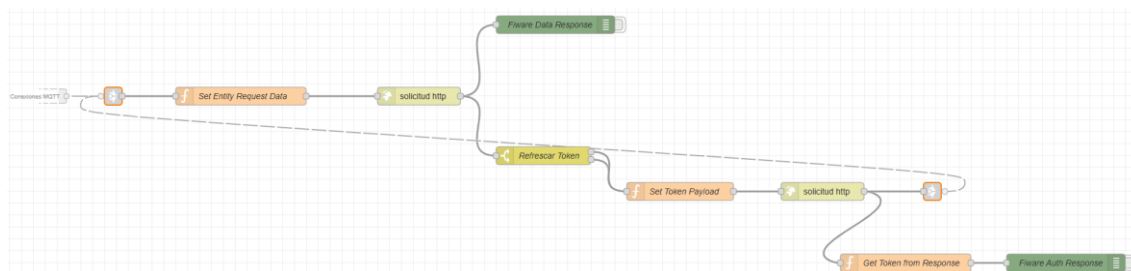


Ilustración 28: Node Red, flujo Conexión Fiware

En la ilustración anterior podemos ver los nodos del flujo mencionado, los cuales se explican a continuación:

- *Set Entity Request Data*: crea la request para enviar la información a Fiware. Escrito en Javascript nos permite definir headers, payload y url (más información en el Anexo 15).
- *Solicitud HTTP*: encontramos la de petición de token y la de POST con envío de la información de las composteras.
- *Selector*: si el código de estado de la petición es 400 o 401, solicitamos de nuevo el token.
- *Set Token Payload*: crea la request para solicitar el token a Fiware. Escrito en Javascript nos permite definir headers, payload y url (más información en el Anexo 16).
- *Get Token from Response*: obtención del token y almacenamiento en una variable de flujo. Escrito en Javascript
- *Módulos de depuración*: para poder verificar las distintas partes del sistema.

Una vez obtenida la información de los sistemas de compostaje, esta será tratada brevemente en el servidor local, para, posteriormente, ser enviada a la base de datos de Diputación de Badajoz utilizando el protocolo Fiware.

No entraremos en detalles del tratamiento y procesado de la información que se llevará a cabo en la plataforma Fiware de la Dip. de Badajoz, ya que de esto se encargará un equipo especializado posteriormente.

En resumen, mostraremos el diagrama de la arquitectura software implementada en el sistema en la siguiente ilustración.

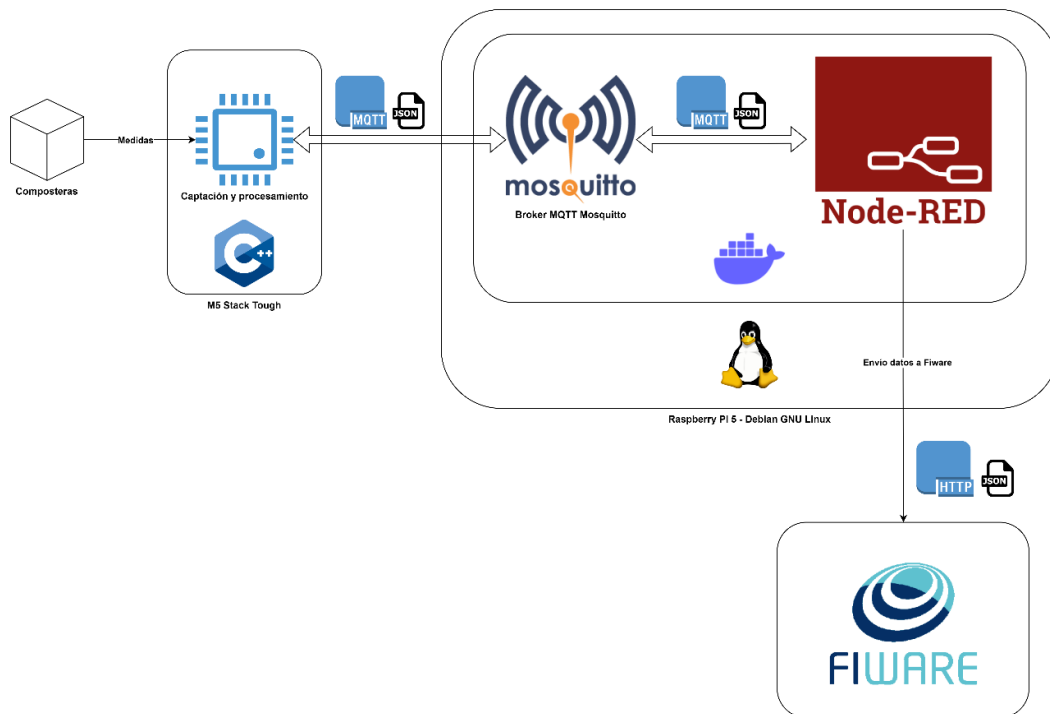


Ilustración 29: Diagrama de arquitectura Software

4.4 Arquitectura de comunicación

4.4.1 Protocolo de comunicación seleccionado

Para este sistema, se ha elegido MQTT (Message Queuing Telemetry Transport) como protocolo principal de comunicación entre los dispositivos desplegados (M5Stack Tough) y el nodo centralizador de datos (Raspberry PI 5).

4.4.2 Topología de red del sistema

Este sistema adopta una tipología de tipo estrella. Esta estructura, reduce la posibilidad de fallo de red conectando todos los nodos a un nodo central.

Cuando se aplica a una red basada en la topología estrella, este concentrador central reenvía todas las transmisiones recibidas de cualquier nodo periférico a todos los nodos periféricos de la red, algunas veces incluso al nodo que lo envió. Todos los nodos periféricos se pueden comunicar con los demás transmitiendo o recibiendo del nodo central únicamente. Un fallo en la línea de conexión de cualquier nodo con el nodo central provocaría el aislamiento de ese nodo respecto a los demás, pero el resto del sistema permanecería intacto [57].

Esto nos proporciona una ventaja fundamental, la alta escalabilidad. Gracias a que podemos añadir nuevos nodos sin necesidad de modificar los que ya existen. Pero no solo hace que nuestro sistema de compostaje sea escalable, sino toda la red de composteras que quiera conectarse a Fiware, es decir, contamos con dos estructuras en estrella:

- Composteras a nodo central local.
- Nodo central local a Fiware.

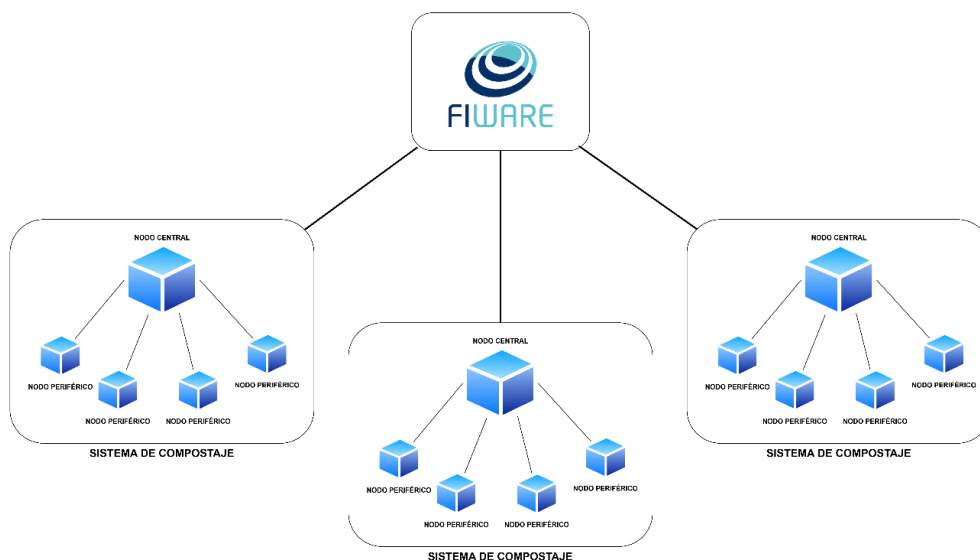


Ilustración 30: Tipología de red en estrella del sistema

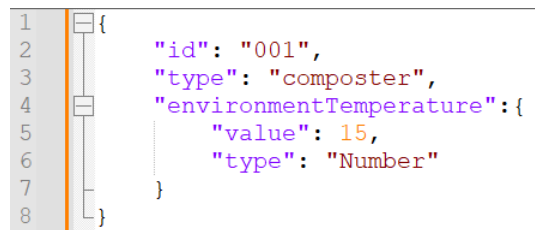
4.4.3 Formato de los mensajes intercambiados

Para la carga útil, hemos utilizado un formato Json. Esto nos permite estandarizar la información a un lenguaje altamente extendido.

Nuestro sistema, cuenta en su última etapa con el envío de datos a Fiware. Este, trabaja con un formato de Json concreto, el estándar NGSI [58], para poder estandarizar el traspaso de información entre dispositivos.

El proyecto, utiliza ese formato en la carga útil desde el principio, es decir, desde la primera comunicación que hacemos desde el M5Stack Tough al bróker a través de MQTT.

Cuando declaramos una entidad en Fiware, esta debe tener tres claves principales: identificador, tipo de entidad y los datos que quedamos enviar subdividido en dos claves, una de valor y otra de tipo.



```
1 {  
2   "id": "001",  
3   "type": "composter",  
4   "environmentTemperature": {  
5     "value": 15,  
6     "type": "Number"  
7   }  
8 }
```

Ilustración 31: Ejemplo formato json para Fiware

Aquí, podemos apreciar los campos mencionados anteriormente:

- *id*: identificador del dispositivo.
- *type*: definición del tipo de entidad.
- *environmentTemperature*: dato que quedamos enviar a fiware.
 - *value*: valor de la variable
 - *type*: tipo de variable

Podemos enviar tantos datos como quedamos en el mismo *payload*, siempre y cuando siga esta estructura.

4.4.4 Broker y gestión de tópicos

Como mencionamos anteriormente, Mosquitto se encuentra desplegado en un contenedor de Docker dentro de nuestro servidor local (Raspberry).

Los mensajes se estructuran utilizando una jerarquía de tópicos que permite organizar la información de forma clara y escalable. En nuestro sistema, únicamente utilizamos dos tópicos; uno de control y otro de mediciones.

- Tópico de mediciones: *'composters/measures'*. En este tópico se publican las mediciones obtenidas de los sensores.

- Tópico de control: *'composters/control'*. Este tópico se utiliza para el control de los sistemas de compostaje; cambios de configuración, información relevante que el sistema necesite transmitir al M5Stack, etc.

Calidad de servicio (QoS)

Se baraja la implementación de dos niveles de QoS en el sistema:

- QoS 1 (al menos una vez): para lecturas periódicas de sensores, donde es preferible asegurar la recepción, aunque implique posibles duplicados.
- QoS 2 (exactamente una vez): reservado para mensajes críticos como alertas o eventos de fallo, donde se busca evitar duplicidades y garantizar la entrega exacta.

Seguridad

Para entornos de pruebas, Mosquitto ha sido configurado en modo local o en red cerrada, sin requerir cifrado ni autenticación avanzada. Sin embargo, admite las siguientes opciones de seguridad que pueden habilitarse para entornos productivos:

- Autenticación por usuario y contraseña, definida en un fichero de claves.
- Cifrado TLS, mediante certificados SSL para asegurar la confidencialidad e integridad de los mensajes.
- Control de acceso (ACLs), para limitar qué tópicos puede publicar o suscribirse cada cliente.

4.4.5 Diagrama de flujo de datos y comunicación

Un diagrama del flujo de datos y el tipo de comunicación se muestra en la siguiente ilustración:

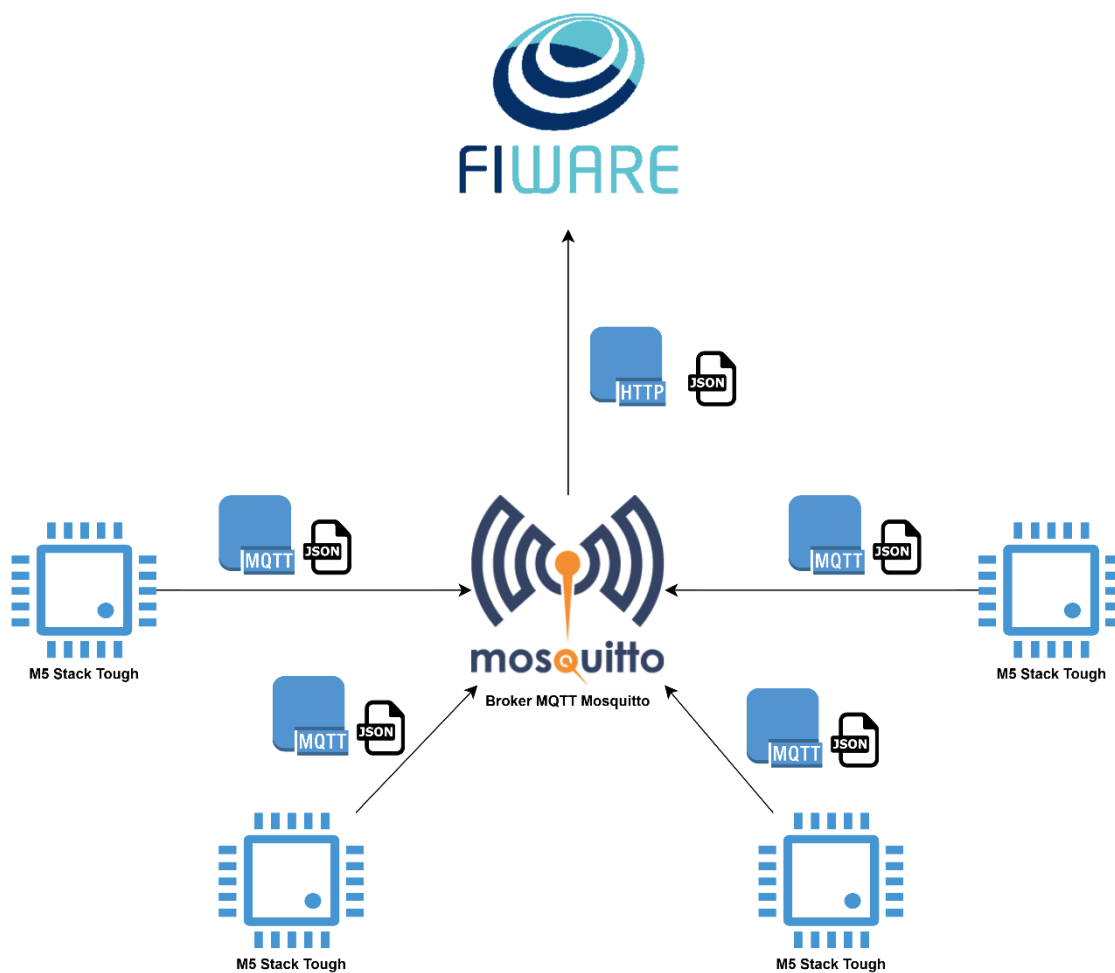


Ilustración 32: Diagrama de comunicación entre dispositivos

4.5 Arquitectura hardware

En este apartado se describe la composición, disposición y funcionamiento del sistema desde el punto de vista del hardware. Se detallan los componentes utilizados, sus conexiones, el por qué de su elección, así como su comportamiento en términos de consumo energético.

4.5.1 Descripción de componentes hardware

En este apartado, nos centraremos en describir los componentes hardware que componen el sistema.

Sistema embebido

Para este sistema, hemos utilizado el dispositivo M5 Stack Tough. Dispositivo versátil y resistente a las exigencias del proyecto.



Ilustración 33: M5Stack Tough [1]

M5 Stack Tough [1] es un controlador embebido programable de grado industrial. Está equipado con el chip de control principal Espressif ESP32, que integra Wi-Fi y cuenta con un microprocesador Xtensa® LX6 de 32 bits de doble núcleo y bajo consumo con una frecuencia principal de hasta 240 MHz. La placa incluye 8 MB de PSRAM + 16 MB de memoria flash, una pantalla IPS a todo color de 2,0 pulgadas y 320x240, un panel táctil capacitivo de alta sensibilidad y diversas interfaces periféricas y de expansión. Su carcasa completamente cerrada ofrece protección contra el polvo y el agua, lo que garantiza un funcionamiento estable del circuito incluso en entornos industriales complejos. Este controlador es ideal para escenarios como control de campo industrial, edificios inteligentes y nodos de adquisición de datos en exteriores.

De igual manera, el diseño puede adaptarse con facilidad a otros modelos basados en ESP32.

Algunas de las principales especificaciones del M5 Stack Tough se muestran en el siguiente listado:

- SoC: ESP32-D0WDQ6-V3, doble núcleo a 240 MHz, 520 KB de SRAM, conectividad Wi-Fi.
- Memoria Flash: 16 MB.
- PSRAM: 8 MB.
- Voltaje de Entrada: USB (5V @ 500 mA) / Entrada de energía vía RS485 (24V @ 1A).
- Interfaces: Tipo-C x1 y Grove x4 (RS485 / I2C / GPIO / UART).
- Pantalla LCD IPS: Tamaño: 2.0", resolución: 320 x 240, modelo: ILI9342C, con pantalla a color, brillo máximo: 853 nits y control Táctil: CHSC6540.
- Altavoces: Amplificador I2S de 16 bits NS4168 + altavoz de 1W.
- Chip RTC: BM8563.
- PMU (Unidad de gestión de energía): AXP192.
- Chip USB: CH9102 (interfaz USB-TTL para programación).
- Ranura para tarjeta TF: Soporta tarjetas de hasta 16 GB.
- Antena: Antena Wi-Fi 3D.
- Material del chasis: Plástico resistente a los rayos UV.
- Dimensiones del producto: 76.0 x 58.0 x 41.6 mm.
- Peso del producto: 117.5 g.
- Tamaño del paquete: 107.0 x 81.4 x 47.7 mm.
- Peso bruto: 151.0 g.
- Precio 60/70€.

Podemos ampliar la información de sus diferentes características utilizando la documentación del fabricante, pero la más relevante se detalla a continuación:

- Desarrollado en base a ESP32, compatible con Wi-Fi
- 16 MB de memoria Flash, 8 MB de PSRAM
- Altavoz incorporado, luz indicadora de encendido, motor de vibración, RTC, amplificador I2S, pantalla táctil capacitiva, botón de encendido, botón de reinicio
- Ranura para tarjeta TF (admite hasta 16 GB)
- Batería de litio incorporada, equipada con un chip de gestión de energía.
- Placa independiente con IMU de 6 ejes incorporada y micrófono PDM
- Interfaz M-Bus

Sensores

Aunque los sensores que se utilicen posteriormente pueden variar de los originales, se detallan una serie de sensores base utilizados para el prototipado del sistema, cumpliendo las especificaciones previas.

ENV IV

Este sensor se utilizará para medir la temperatura y humedad del interior de la compostera. Cabe precisar que no es la misma medida que la temperatura y humedad en el interior de la materia orgánica.



Ilustración 34: Sensor ENV IV [59]

La unidad ENV-IV [59] es un sensor ambiental con sensores SHT40 y BMP280 integrados para medir temperatura, humedad y presión atmosférica. El SHT40 es un sensor digital de temperatura y humedad de alta precisión y bajo consumo compatible con la interfaz I²C (dirección I²C del SHT40: 0x44). El BMP280 es un sensor de presión absoluta que proporciona mediciones de presión atmosférica de alta precisión (dirección I²C del BMP280: 0x76). Es adecuado para la monitorización meteorológica, la monitorización del ambiente interior, la agricultura y la horticultura, y la automatización industrial.

Para desarrollar las especificaciones de forma más detallada y ordenada nos apoyamos en la siguiente tabla.

- Rango de medidas del SHT40: -40 ~ 125 °C, 0 ~ 100 %HR.
- Rango de medidas del BMP280: 300 ~ 1100 hPa.
- Precisión del SHT40: ± 0.2 °C, ± 1.8 %HR.
- Precisión del BMP280: ± 0.12 hPa.
- Protocolo de comunicación: I2C, SHT40 con dirección 0x44 y BMP280 con dirección 0x76.
- Material de la carcasa: Plástico (PC).
- Tamaño del producto: 32.0 x 24.0 x 8.0 mm.
- Peso del producto: 3.9 g.
- Tamaño del paquete: 138.0 x 93.0 x 9.0 mm.
- Peso bruto: 9.4 g.
- Precio 10/15€.

ENS160+AHT21

Utilizaremos este sensor para medir el nivel de CO₂ del interior de la compostera.

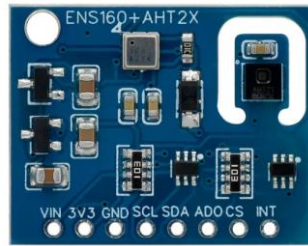


Ilustración 35: Sensor ENS160+AHT2X [60]

Es un sensor digital multigas, especialmente diseñado para el control de la calidad del aire en interiores. Es capaz de detectar una gran variedad de gases y compuestos orgánicos volátiles (COV), como tolueno, hidrógeno, etanol, NO₂ y ozono [61].

Las principales características de este sensor se describen a continuación [61]:

- Detección de gases reductores (COV) y oxidantes (como NO₂).
- MOX admite hasta 4 sensores de gas independientes.
- Medición de sensor integrada y control de accionamiento del calefactor.
- Fusión de sensor precalibrada integrada y algoritmo de corrección automática de la línea de base.
- Múltiples salidas IAQ (TVOC, ecO₂, AQL).
- Interfaz I2C y SPI.
- < 1 minuto de calentamiento.
- < 1 hora de arranque.
- Alta resistencia al siloxano y a la humedad.
- Facilidad de uso. Selección de señal de calidad del aire multiestándar.
- Fácil integración. No se requiere ninguna biblioteca de software, que no tiene impacto en la CPU del host.
- Ambiente - 40 a + 85 ° C / 5 t.
- Precio 15/20 €.

DS18B20

Utilizamos este sensor para medir la temperatura dentro de la materia orgánica.



Ilustración 36: Sensor DS12B20 [62]

El termómetro digital DS18B20 [63] proporciona lecturas de temperatura de 9 a 12 bits (configurables) que indican la temperatura del dispositivo. La información se envía a el DS18B20 utilizando una interfaz 1-Wire, por lo que sólo es necesario conectar un cable (y tierra) desde un microprocesador central a un DS18B20. La alimentación para leer, escribir y realizar conversiones de temperatura se puede derivar de la propia línea de datos sin necesidad de una fuente de alimentación externa.

Dado que cada DS18B20 contiene un número de serie de silicio único, pueden existir varios DS18B20 en el mismo bus 1-Wire. Esto permite colocar sensores de temperatura en muchos lugares diferentes. Entre las aplicaciones en las que esta característica resulta útil se incluyen los controles ambientales de HVAC, la detección de temperaturas en el interior de edificios, equipos o maquinaria, y la supervisión y control de procesos.

Algunas de las características principales de este sensor, son:

- Interfaz única 1-Wire que requiere solo un pin de puerto para la comunicación.
- Capacidad multidrop que simplifica las aplicaciones de detección de temperatura distribuidas.
- No requiere componentes externos.
- Puede alimentarse a través de la línea de datos. El rango de alimentación es de 3.0V a 5.5V.
- No requiere energía en modo de espera.
- Mide temperaturas desde -55°C hasta +125°C (equivalente en Fahrenheit: -67°F a +257°F).
- Precisión de $\pm 0.5^{\circ}\text{C}$ entre -10°C y +85°C.
- Resolución del termómetro programable de 9 a 12 bits.
- Convierte temperatura de 12 bits a palabra digital en 750 ms (máximo).
- Configuración de alarmas de temperatura no volátil y definida por el usuario.
- El comando de búsqueda de alarma identifica y direcciona dispositivos cuya temperatura está fuera de los límites programados (condición de alarma de temperatura).
- Precio 3/5€.

Ph0-14 + Sonda

Utilizamos este sensor para la medición de PH [64]de la materia orgánica dentro del contenedor de compostaje.



Ilustración 37: Sensor PH0-14 + Sonda [65]

Este sistema permite medir el nivel de pH de soluciones líquidas en tiempo real, utilizando una placa Arduino, una sonda de pH estándar tipo electrodo y un módulo acondicionador de señal compatible con rangos de pH de 0 a 14. Es ideal para proyectos de automatización, control de calidad de agua, hidroponía, acuarios, laboratorios y sistemas de cultivo.

Algunas de las características principales de este sensor son:

- Potencia del Módulo : 5.00V
- Tamaño del Módulo : 43 x 32mm(1.69×1.26")
- Rango de Medición :0 - 14PH
- Temperatura de Medición: 0 - 60 2C
- Precisión : $\pm 0.1\text{pH}$ (25 °C)
- Tiempo de respuesta : $\leq 1\text{min}$
- Sensor de pH con conector BNC
- Interfaz pH2.0 (3 foot patch)
- Potenciómetro de ajuste de ganancia
- LED indicador de encendido.
- Precio 20/30€.

Servidor local

Para alojar el servidor local, hemos utilizado una Raspberry PI 5 [66]. Este dispositivo cuenta con la potencia necesaria para poder mantener la infraestructura local sin problemas, además, su uso es muy extendido por lo que nos aporta facilidad de compatibilidad y resolución de problemas.

Raspberry PI 5



Ilustración 38: Raspberry PI 5 [66]

Con un procesador Arm Cortex-A76 de cuatro núcleos y 64 bits que funciona a 2,4 GHz, Raspberry Pi 5 ofrece un aumento del rendimiento de la CPU de 2 a 3 veces con respecto a Raspberry Pi 4. Junto con una mejora sustancial del rendimiento gráfico de gracias a una GPU VideoCore VII de 800 MHz, una salida de pantalla dual 4Kp60 a través de HDMI y la compatibilidad con cámaras de última generación de un procesador de señal de imagen Raspberry Pi rearquitectado, ofrece una experiencia de escritorio fluida para los consumidores y abre la puerta a nuevas aplicaciones para los clientes industriales.

Algunas de las características más importantes de este modelo de Raspberry Pi son [66]:

- CPU Broadcom BCM2712 de cuatro núcleos a 2,4 GHz y 64 bits Arm Cortex-A76, con extensiones criptográficas, cachés L2 de 512 KB por núcleo y una caché L3 compartida de 2 MB.
- GPU VideoCore VII, compatible con OpenGL ES 3.1, Vulkan 1.2.
- Doble salida de pantalla HDMI® 4Kp60 compatible con HDR.
- Decodificador HEVC 4Kp60.
- SDRAM LPDDR4X-4267 (2 GB, 4 GB, 8 GB y 16 GB).
- Wi-Fi® 802.11ac de doble banda.
- Bluetooth 5.X de alta velocidad / Bluetooth Low Energy (BLE) 0 / Bluetooth Low Energy (BLE).
- Ranura para tarjetas microSD, compatible con el modo SDR104 de alta velocidad.
- 2 × puertos USB 3.0, compatibles con funcionamiento simultáneo a 5 Gbps.
- 2 × puertos USB 2.0.
- Gigabit Ethernet, compatible con PoE+ (requiere HAT PoE+ independiente)
- 2 × transceptores MIPI de 4 carriles para cámara/pantalla.

- Interfaz PCIe 2.0 x1 para periféricos rápidos (requiere M.2 HAT u otro adaptador por separado).
- Alimentación de CC de 5 V/5 A a través de USB-C, compatible con Power Delivery.
- Cabecera de 40 patillas estándar de Raspberry Pi.
- Reloj en tiempo real (RTC), alimentado por batería externa.
- Botón de encendido.
- Precio 80/100€.

Fuente de alimentación

Como es de suponer, todo este sistema necesita alimentación eléctrica para poder funcionar. Aunque esta infraestructura se detallará más adelante en el apartado de despliegue, podemos dividir la alimentación pensada para el sistema en dos variantes.

Cableado directo

En este caso, contaríamos con una toma de luz disponible cerca de los dispositivos, lo que permitiría alimentarlos de forma ininterrumpida.

Baterías

En casos de difícil acceso a toma de luz directa, el sistema puede ser implementado con baterías de pocos amperios e incluso con opción de recarga solar. Esto es debido al bajo consumo del sistema y los ahorros de energía que encontramos en los modos de ‘reposo’ de los dispositivos.

4.5.2 Esquema de conexiones y puertos

Durante este apartado, explicaremos el esquema de conexiones físicas que se ha seguido para llevar a cabo el montaje del sistema hardware. Dado que los distintos sensores utilizan diferentes protocolos de comunicación, se ha dividido este apartado en función del tipo de interfaz empleada.



Ilustración 39: Puertos de conexión del M5Stack Tough [1]

Conexiones I2C

Sensores que se comunican mediante el bus I2C del sistema embebido. Del dispositivo necesitamos los pines GND, VCC, SDA y SCL. El M5Stack Tough cuenta con un puerto dedicado a esta comunicación de tipo HY2.0-4P. Llamado I2C o PORT A (terminal rojo).

HY2,0-4P	Negro	Rojo	Amarillo	Blanco
PORT A	GND	5V	G32	G33

Ilustración 40: Pinout PORT A M5Stack Tough

El esquema de conexión utilizado con los sensores es el siguiente:

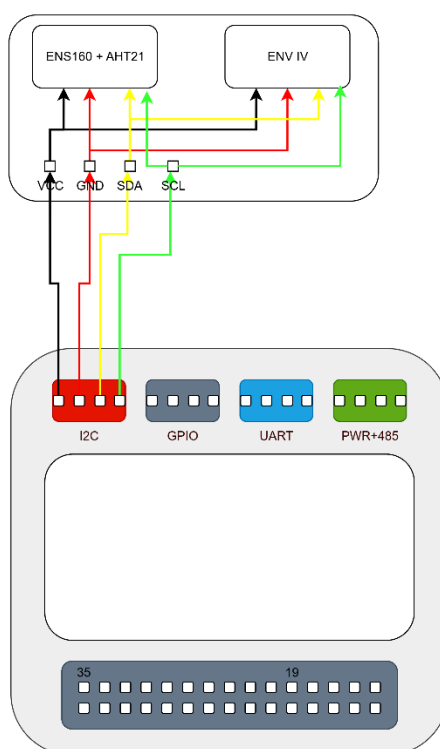


Ilustración 41: Conexiones I2C M5Stack Tough

Conexiones 1Wire

Sensores que se comunican de forma 1Wire con nuestro M5Stack Tough, en este caso, el único sensor que utiliza esta comunicación es el de medición de temperatura del compost (DS18B20).

Para esto, necesitamos los pines GND, VCC (5V) y el pin GPIO19 de la placa. De esta forma, iniciando la conexión 1Wire, podemos poner el número de sensores que consideremos oportunos en un mismo bus.

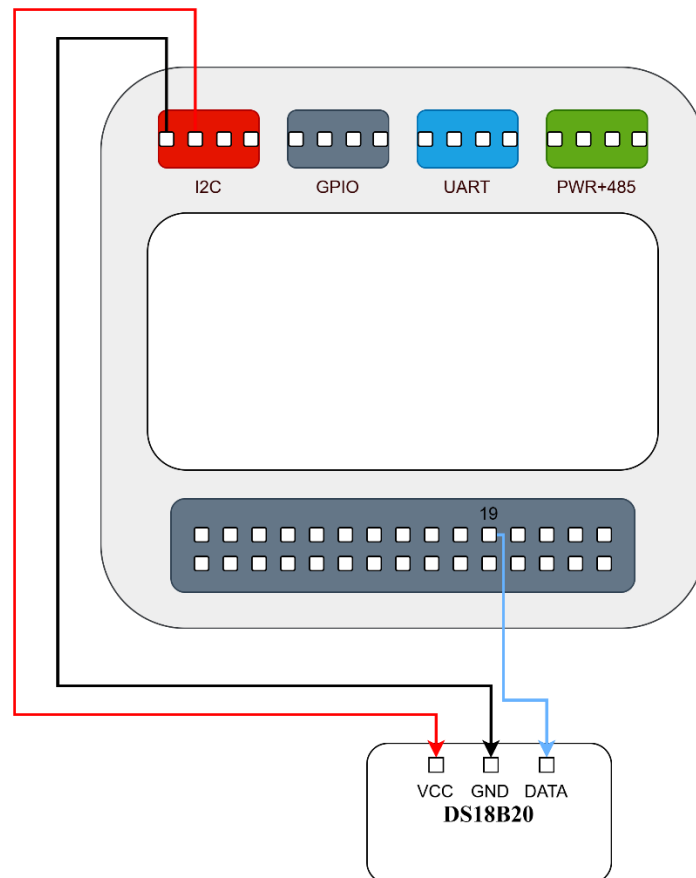


Ilustración 42: Conexión 1Wire M5Stack Tough

Conexiones Analógicas

Aquí nos encontramos con los sensores que tomarán medidas de voltaje analógico. El sensor que hasta el momento utiliza este tipo de conexión es el medidor de Ph.

Siguiendo con el esquema genera, necesitamos utilizar pines GND, VCC para alimentación y el pin GPIO35

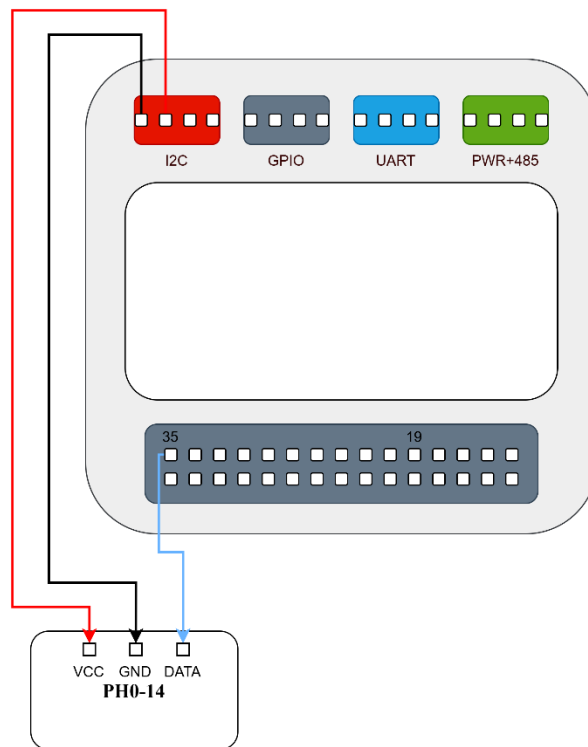


Ilustración 43: Conexiones analógicas M5Stack Tough

4.5.3 Consumo energético y autonomía

Considerando los componentes hardware mencionados previamente, podemos hacer una estimación de consumo teórica, analizando sus hojas de características y el tiempo promedio en que estarán encendidos a lo largo del día. Si agrupamos estos valores en una tabla, tenemos lo siguiente.

Componente	Consumo típico (mA)	Tiempo activo estimado (h/día)	Consumo diario (mAh)
M5Stack Tough (activo)	80 mA	2 h	160 mAh
M5Stack Tough (deep sleep)	~0.15 mA	22 h	3.3 mAh
PH0-14 sensor	~5-10 mA	2 h	15 mAh
DS18B20	1.5 mA durante lectura	0.1 h (6 min)	0.15 mAh
ENS160 (VOC sensor)	10.8 mA	2 h	21.6 mAh
AHT21 (T/H)	0.8 mA (standby), 1.9 mA (medición)	2 h	~1.9 mAh
ENV IV Unit (SHT40, QMP6988)	~1.5 mA combinados	2 h	3 mAh

Tabla 1: Consumo medio del sistema para 2h de funcionamiento

Se ha estimado una media de 2 horas al día de completo funcionamiento del sistema, el resto del tiempo se encontrará en reposo para ahorrar energía.

En el caso de que el sistema dependa de baterías, podemos hacer una estimación de la batería necesaria y el tiempo de recambio.

Parámetro	Valor
Consumo diario estimado	204.95 mAh
Capacidad batería (LiPo)	2500 mAh
Autonomía sin recarga	~12.2 días

Tabla 2: Autonomía del sistema con batería

Sin embargo, también podemos contar con sistemas de recarga solares. Si suponemos un panel solar de 5V/1W con una eficiencia efectiva de 200mA durante 5 horas de sol al día:

- **Carga diaria**
 $200 \text{ mA} \times 5 \text{ h} = 1000 \text{ mAh/día}$
- **Balance energético diario**
 $1000 \text{ mAh (generados)} - 205 \text{ mAh consumidos} = +795 \text{ mAh/día}$

Esto garantizaría un funcionamiento indefinido y buen dimensionamiento del circuito de carga. Esta posibilidad es altamente recomendable ya que en Extremadura contamos con muchas horas de sol anuales [67].

4.5.4 Diagrama de arquitectura hardware

Para finalizar, en la siguiente ilustración se muestra el diagrama completo de la arquitectura hardware propuesto para el sistema de compostaje.

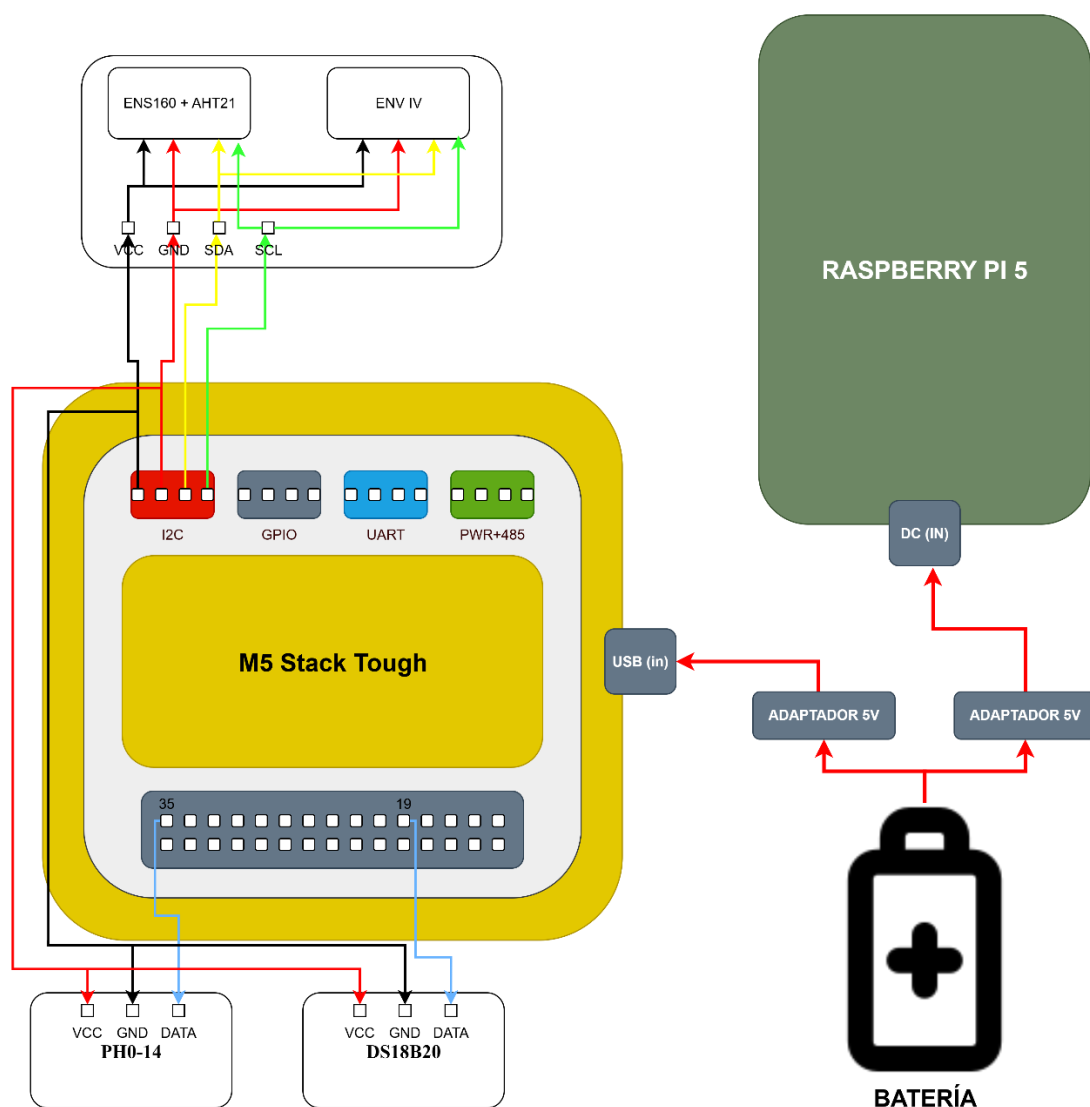


Ilustración 44: Diagrama de arquitectura hardware

4.6 Despliegue y montaje físico

4.6.1 Entorno de instalación

Condiciones ambientales

Debido a la configuración física elegida para montaje, el sistema no se considera comprometido por los requisitos medioambientales.

Los dispositivos sensibles a la humedad contarán con cierres aislados además de registros con protección estanca. Sin embargo, es importante mencionar que los sensores que se encuentren dentro de la zona del compost si se verán afectados por las condiciones del proceso de compostaje.

Requisitos físicos

A continuación, se describen las principales necesidades de energía, red y seguridad entre otras para la infraestructura:

Energía

- Punto de luz tipo Schuko (Tipo F) de 230VAC.
- Batería de respaldo (UPS) para cortes en el suministro habitual.

Conectividad

Se plantean dos tipos de montaje: **inalámbrico** o **semi-cableado**.

- **Inalámbrico:** la zona de montaje contará con conexión Wifi activa, accesible y segura para el sistema. Tanto el sistema central como los controladores estarán conectados de forma inalámbrica utilizando esta red.
- **Semi-Cableado:** el sistema central contará con conexión Ethernet y los controladores se conectarán mediante Wifi a este. Para esta casuística podemos contar con el caso anterior (inalámbrico) o desplegar una pequeña red sin conexión externa, únicamente para conectar controladores y sistema central.

Protección del sistema

Condiciones ambientales

- Registros estancos para alojar el sistema central y los controladores.
- Controladores con protecciones IP y conexiones estancas

Seguridad vandálica

- Los registros contarán con llave para su apertura y manipulación únicamente por personal autorizado.
- Los registros contarán con material resistente frente a golpes, arañados y demás daños que puedan provocarse de forma intencionada o accidental.

4.6.2 Montaje e instalación

Para poder explicar de forma más clara el montaje físico del sistema utilizaremos el ejemplo de un sistema de compostaje con 3 contenedores de residuos iguales, los cuales se aprecian en la siguiente ilustración.

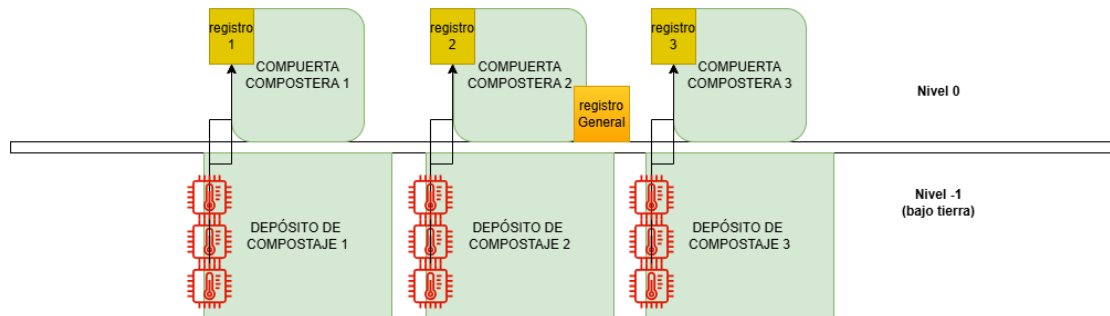


Ilustración 45: Diagrama de montaje general

Distribución principal

Cada sistema de compostaje estará formado por un número definido de composteras (a especificar en función de los requerimientos). Además, contamos con un registro general en el que albergaremos el servidor local y varios registros secundarios (tantos como composteras haya en el sistema) donde almacenaremos los controladores.

Las composteras, se encuentra dividida en dos partes:

- Compuerta: zona de acceso al compost para su deposición o manipulación.
- Depósito: zona de almacenamiento del compost orgánico. Esta se localiza bajo el nivel del suelo.

Cada compostera cuenta con unos sensores propios para la medición de los valores necesarios, además de un M5 Stack Tough como controlador.

Los sensores se distribuirán dentro del propio depósito, situando algunos en la parte superior, alejados del compost (sensores de entorno) y otros en la parte inferior, cerca o dentro del compost (sensores de biomasa).

Los registros secundarios se encargarán de almacenar los componentes físicos de cada compostera que se encuentren fuera de la zona de materia orgánica, es decir, el controlador y el sistema de alimentación auxiliar (UPS) propio (si fuera necesario).

El registro principal se encargará de almacenar el servidor local (Raspberry) y su sistema de alimentación auxiliar (UPS) (si fuera necesario).

A continuación, se muestra una ilustración donde puede apreciarse la distribución de los componentes de las composteras.

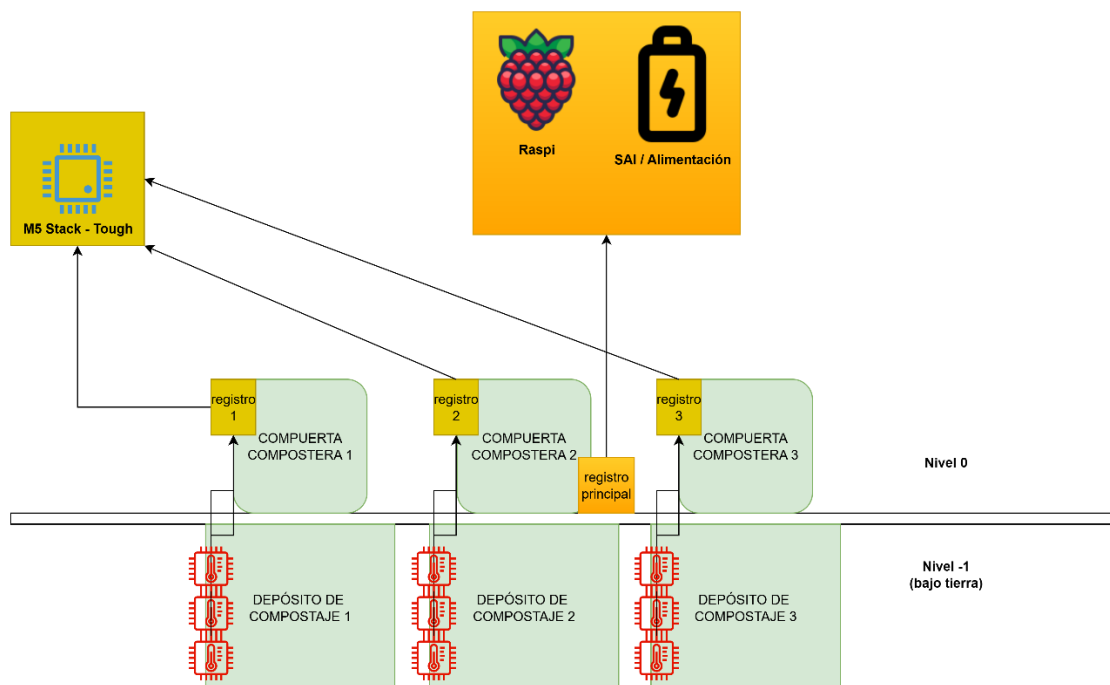


Ilustración 46: Distribución de componentes en las composteras

Sistema de conexiones eléctricas

Las conexiones eléctricas podemos dividir las principalmente en tres:

- Canalización principal

Esta es la línea de alimentación principal del sistema. Consiste en una toma general de 220V que va desde el exterior de la instalación hasta el registro principal. Además, si el sistema es de tipo semi-cableado, esta tirada puede estar también constituida por un cableado de ethernet. En el esquema se representa en color rosa.

- Canalización secundaria

Esta línea es la que alimentará los controladores. Está formada por el cableado que se extiende desde el registro principal a los registros secundarios. En el esquema se represente en color verde.

- Canalización terciaria

Esta línea se encarga de conectar los sensores situados en la compostera con su controlador en el registro secundario correspondiente. Esta tirada llevará tanto cableado de datos como de alimentación. En el esquema se representa de color rojo.

A continuación, se muestra una ilustración con la distribución de las conexiones eléctricas utilizadas.

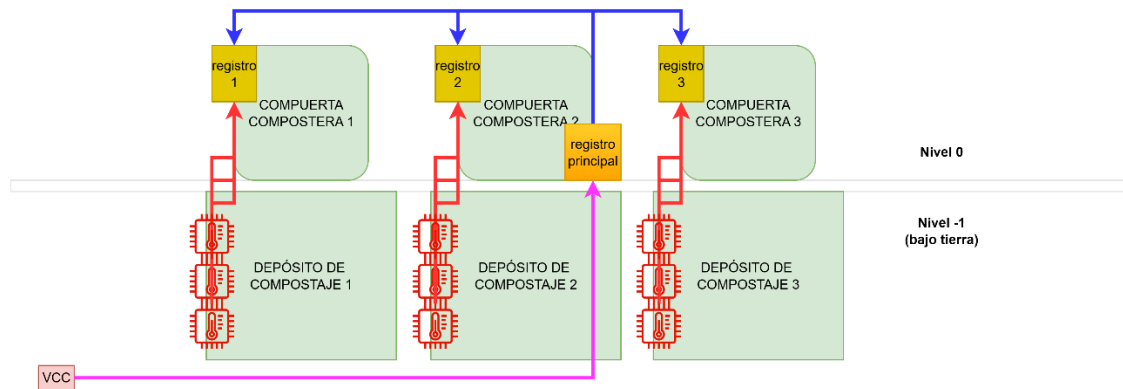


Ilustración 47: Distribución de conexiones eléctricas

4.6.3 Análisis de costes y tiempo de despliegue

En este apartado, explicaremos detalladamente el coste que supondría montar el sistema físico en un espacio real. Además, haremos una aproximación del tiempo necesario para su despliegue.

Estimación de costes y disponibilidad

Para un correcto montaje del sistema, tal y como está diseñado, el coste oscilaría entre los 225 y los 295€. Esta variación es debida a los diferentes precios según los distribuidores. Para la estimación se han elegido rangos de precio medios.

En la siguiente tabla se muestra el desglose de precios de cada componente:

Componente	Precio estimado (EUR)
M5Stack Tough	60–70 €
Raspberry Pi 5 (8GB)	80–100 €
Batería LiPo (power bank 20Ah o similar con salida 5V/2A)	25–35 €
Sensor ENV IV (temp., humedad, presión)	10–15 €
Sensor ESN160 + AHT121 (CO ₂ + temperatura/humedad)	15–20 €
Sensor DS18B20 (temperatura suelo, impermeable)	3–5 €
Sensor de pH 0–14 + sonda	20–30 €
Cables, conectores, encapsulado adicional	10–20 €

Tabla 3: Estimación de costes materiales

Todos los componentes utilizados son accesibles en el mercado actual, el más crítico podría considerarse la Raspberry Pi 5 debido a su alta demanda. En la siguiente tabla se puede observar una estimación de disponibilidad de los componentes mencionados anteriormente.

Componente	Disponibilidad	Observaciones
M5Stack Tough	Media – Alta	Se encuentra en distribuidores como Mouser o Amazon, pero puede haber roturas de stock puntuales.
Raspberry Pi 5	Media	Puede haber problemas de disponibilidad o precios inflados según la demanda. Se recomienda compra anticipada.
Batería (power bank)	Alta	Fácilmente disponible en tiendas de electrónica o consumo.
Sensor ENV IV	Alta	Compatible con M5Stack; buena disponibilidad en tiendas oficiales.
ESN160 + AHT121	Media	Más difícil de encontrar que otros sensores; considerar alternativas en caso de stock limitado.

DS18B20	Alta	Sensor muy común y económico; múltiples variantes disponibles.
Sensor pH + sonda	Media	Suele estar disponible, pero las sondas baratas tienen menor durabilidad; puede haber variabilidad en calidad.

Tabla 4: Estimación de disponibilidad de materiales

Estimación de tiempos

Para poder dimensionar correctamente el despliegue, es importante conocer de forma aproximada el tiempo necesario para poner en funcionamiento el sistema.

El tiempo total estimado para estas tareas puede variar entre 19 y 27 horas. Se aplican márgenes para posible solución de errores y complicaciones en el montaje. A continuación, se muestra una tabla de distribución de tiempos en función de las tareas.

Tarea	Tiempo estimado
Montaje físico de sensores y conexiones	4–6 horas
Configuración y prueba de la Raspberry Pi	2–3 horas
Instalación de broker MQTT y servicios locales	2–3 horas
Programación de firmware en M5Stack Tough	4–6 horas
Integración con FIWARE (IoT Agent + Context Broker)	4–5 horas
Pruebas de comunicación y validación de datos	3–4 horas

Tabla 5: Distribución de tiempos para despliegue

6 Resultados

A lo largo del desarrollo del trabajo, se han alcanzado con éxito todos los objetivos propuestos inicialmente. A continuación, se detallan los resultados obtenidos, estructurados conforme a los objetivos marcados.

- **Creación de un sistema de compostaje inteligente**

Se ha diseñado e implementado un sistema de compostaje inteligente, capaz de monitorizar y reportar en tiempo real parámetros clave del proceso de compostaje, como temperatura, humedad o lixiviados. Los sensores conectados a microcontroladores recopilan estos datos, los procesan localmente y los transmiten mediante el protocolo MQTT. Este sistema permite un seguimiento automatizado del estado de las composteras, optimizando la gestión del compost y facilitando la toma de decisiones.

- **Interconexión y centralización mediante MQTT**

La comunicación entre dispositivos y la infraestructura central se ha establecido utilizando el protocolo MQTT, lo que ha permitido una transmisión ligera, rápida y eficiente de los datos generados. Se ha implementado un broker MQTT profesional que actúa como punto central de comunicación, asegurando la escalabilidad del sistema y su estabilidad frente a múltiples conexiones simultáneas. Todos los nodos activos del sistema están conectados a este broker, facilitando una gestión centralizada.

- **Conexión con la base de datos de Diputación de Badajoz mediante FIWARE**

Los datos recopilados y centralizados se integran con la plataforma Fiware, promovida por la Diputación de Badajoz, mediante el uso de componentes como el Orion Context Broker. La información se publica en formato NGSI y se actualiza de forma continua, lo que permite su visualización, análisis y explotación por parte de la entidad pública. Esto proporciona una solución alineada con los objetivos de digitalización e innovación en la gestión ambiental.

- **Infraestructura robusta con gestión de errores**

El sistema incorpora mecanismos de tolerancia a fallos y gestión de errores, incluyendo reconexiones automáticas al broker MQTT en caso de pérdida de conexión, verificación de integridad en los datos transmitidos y persistencia local en caso de fallo de red. Esta robustez asegura un funcionamiento continuo incluso en entornos poco estables, garantizando la disponibilidad y fiabilidad de la información.

- **Escalabilidad**

Uno de los aspectos clave en el diseño ha sido la escalabilidad. La arquitectura modular y el uso de protocolos estándar como MQTT y HTTP permiten integrar nuevas unidades de compostaje sin necesidad de reconfigurar el sistema existente. Además, el diseño del backend y el sistema de integración con Fiware permiten añadir nuevos dispositivos sin afectar al rendimiento ni a la estructura general.

- **Facilidad de despliegue y modificación**

Se han utilizado tecnologías y metodologías que facilitan la instalación y modificación del sistema. El uso de contenedores y scripts de automatización permite desplegar rápidamente tanto la infraestructura central como los nodos individuales. La documentación generada y la estructura del código aseguran que el sistema puede ser actualizado o adaptado en función de nuevas necesidades, sin requerir modificaciones profundas.

7 Conclusions

This work has successfully developed and implemented a smart, interconnected, and scalable composting system, fulfilling all the objectives defined at the beginning of the project. The proposed solution not only demonstrates the technical feasibility of applying IoT technologies and open platforms such as FIWARE in the management of organic waste, but also lays the foundation for its potential large-scale deployment in real-world environments such as residential communities or educational centers.

The choice of protocols such as MQTT and HTTP has enabled efficient and robust communication between the deployed devices and the central server. As a result, real-time monitoring of key composting parameters has been achieved, improving both process control and traceability.

The integration with the FIWARE platform has provided significant added value, facilitating interoperability with other solutions within the framework of smart cities and regions. Furthermore, the modular design and infrastructure prepared for error management have endowed the system with high resilience and ease of maintenance.

Overall, the project has demonstrated that it is possible to develop a sustainable technological infrastructure that is replicable and adaptable to different contexts.

Ultimately, this work contributes to the digitalization of environmental processes, aligning with the sustainability and innovation goals promoted by public entities such as Diputación de Badajoz, and showcases the potential of open technologies to transform waste management in an intelligent context.

8 Trabajos Futuros

A partir de los resultados obtenidos, se identifican varias líneas de trabajo futuras que permitirían ampliar, optimizar y evolucionar la solución desarrollada:

Implantación real del sistema

Se plantea la posibilidad de desplegar el sistema en puntos reales de compostaje, tanto en entornos urbanos como rurales. Gracias a la escalabilidad de la arquitectura, sería posible integrar nuevos nodos sin necesidad de reconfigurar la infraestructura existente.

Monitorización avanzada del proceso de compostaje

Se podrían incorporar sensores adicionales para medir variables como:

- Emisión de gases para una evaluación más precisa del estado del compost.
- Volumen de material o nivel de llenado del compostador.

Análisis predictivo e inteligencia artificial

La integración de técnicas de aprendizaje automático podría permitir:

- Predecir el tiempo óptimo de maduración del compost.
- Detectar comportamientos anómalos en los datos (por ejemplo, fermentaciones inadecuadas).
- Ajustar automáticamente los parámetros de gestión para mejorar el proceso.

Desarrollo de interfaces para usuarios finales

Podría desarrollarse una plataforma web o aplicación móvil que permita:

- Consultar el estado de los compostadores en tiempo real.
- Recibir notificaciones ante incidencias.
- Acceder a estadísticas y gráficas de evolución.

Integración con sistemas municipales de gestión de residuos

La información generada por el sistema puede vincularse con los sistemas de recogida selectiva de residuos orgánicos u otras políticas medioambientales locales, aportando datos útiles para la toma de decisiones y optimización de recursos.

Certificación y normalización del sistema

Con vistas a una posible explotación comercial o institucional, sería interesante analizar los estándares y normativas aplicables al sistema, así como evaluar su certificación como solución tecnológica dentro de iniciativas *smart region* o *smart city*.

9 Bibliografía

- [1] M5Stack, «Tough,» 2021. [En línea]. Available: <https://docs.m5stack.com/en/core/tough>. [Último acceso: Junio 2025].
- [2] Fiware Space, «Fiware Space,» 2018. [En línea]. Available: <https://fiware.space/>. [Último acceso: Junio 2025].
- [3] Fiware, «European Commission Encourages Open Source Use Through Fiware,» 2020. [En línea]. Available: <https://www.fiware.org/2020/10/28/european-commission-encourages-open-source-use-through-fiware/>. [Último acceso: Junio 2025].
- [4] Pestrol, «The Cycle of Growth: Understanding the Composting Process,» 2024. [En línea]. Available: <https://www.pestrol.com.au/the-cycle-of-growth-understanding-the-composting-process/>. [Último acceso: Junio 2025].
- [5] Wikipedia, «Compost,» 2025. [En línea]. Available: <https://en.wikipedia.org/wiki/Compost>.
- [6] Naturbag, «Naturbag,» 2022. [En línea]. Available: <https://naturbag.com/home-composting-vs-industrial-composting/>. [Último acceso: Junio 2025].
- [7] EPA, «Approaches Composting,» 2025. [En línea]. Available: <https://www.epa.gov/sustainable-management-food/approaches-composting>.
- [8] K. Nirosika, P. Dulanja, A. Deshani, R. Tang, Y. Cai y S. Chang, «Converting food waste into soil amendments for improving soil sustainability and crop productivity: A review,» 2023. [En línea]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0048969723019307>. [Último acceso: Junio 2025].
- [9] N. M. Sunar, E. I. Stentiford, D. I. Stewart y L. A. Fletcher, «THE PROCESS AND PATHOGEN BEHAVIOUR IN COMPOSTING,» 2009. [En línea]. Available: <https://arxiv.org/pdf/1404.5210>. [Último acceso: Junio 2025].
- [10] Wikipedia, «Organic Fertilizer,» 2021. [En línea]. Available: https://en.wikipedia.org/wiki/Organic_fertilizer. [Último acceso: Junio 2025].
- [11] EPA, «Composting at home,» 2025. [En línea]. Available: <https://www.epa.gov/recycle/composting-home>. [Último acceso: Junio 2025].
- [12] M. Dąbrowska, «How IoT is transforming yields and optimising resources in agriculture,» 2024. [En línea]. Available: <https://www.iot->

- now.com/2024/10/30/147619-how-iot-is-transforming-yields-and-optimising-resources-in-agriculture/. [Último acceso: Junio 2025].
- [13] Insidetelecom, «The new frontier of smart farming powered by 5G,» 2020. [En línea]. Available: <https://insidetelecom.com/the-new-frontier-of-smart-farming-powered-by-5g/>. [Último acceso: Junio 2025].
 - [14] I. Diestro, «Implementación de un espacio de trabajo interconectado con dispositivos activos que se comunican utilizando el protocolo MQTT,» Noviembre 2021. [En línea]. Available: https://smartopenlab.com/storage/2021/12/MemoriaSubida_TFG_GISIT747_2021.pdf.
 - [15] I. Ungurean y N. Gaitan, «A Software Architecture for the Industrial Internet of Things—A Conceptual Model,» 2020. [En línea]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7583771/>. [Último acceso: Junio 2025].
 - [16] A. Xavier, «MQTT Protocol in IoT: Revolutionizing Device Communication,» 2024. [En línea]. Available: <https://www.cavliwireless.com/blog/nerdiest-of-things/what-is-the-mqtt-protocol>. [Último acceso: Junio 2025].
 - [17] A. Oeder, «IoT-Bus – The Secure Communication Bus,» 2020. [En línea]. Available: <https://www.iis.fraunhofer.de/en/ff/lv/net/iot-bus.html>. [Último acceso: Junio 2025].
 - [18] C. Bormann, A. Castellani y Z. Shelby, «CoAP: An Application Protocol for Billions of Tiny Internet Nodes,» 2012. [En línea]. Available: <https://ieeexplore.ieee.org/document/6159216>. [Último acceso: Junio 2025].
 - [19] PickData, «MQTT vs CoAP, la batalla por ser el mejor protocolo IoT,» 2022. [En línea]. Available: <https://www.pickdata.net/es/noticias/mqtt-vs-coap-mejor-protocolo-iot>. [Último acceso: Junio 2025].
 - [20] S. Vinoski, «Advanced Message Queuing Protocol,» 2006. [En línea]. [Último acceso: Junio 2025].
 - [21] Wallarm, «Wallarm,» 2021. [En línea]. Available: <https://www.wallarm.com/what/what-is-amqp>. [Último acceso: Junio 2025].
 - [22] S. Quincozes, T. Emilio y J. Kazienko, «MQTT Protocol: Fundamentals, Tools and Future Directions,» 2019. [En línea]. Available: <https://ieeexplore.ieee.org/document/8931137>. [Último acceso: Junio 2025].
 - [23] K. Tomov, «Learn How to Use The MQTT Protocol in Delphi With This Library,» 2021. [En línea]. Available: <https://blogs.embarcadero.com/learn-how-to-use-the-mqtt-protocol-in-delphi-with-this-library/>. [Último acceso: Junio 2025].

- [24] E. Curry, D. Chambers y G. Lyons, «A JMS Message Transport Protocol for the JADE platform,» 2003. [En línea]. Available: <https://ieeexplore.ieee.org/document/1241153>. [Último acceso: Junio 2025].
- [25] L. Gupta, «JMS Tutorial – Java Message Service Tutorial,» 2021. [En línea]. Available: <https://howtodoinjava.com/jms/jms-java-message-service-tutorial/>. [Último acceso: Junio 2025].
- [26] J. Schlesselman, G. Pardo-Castellote y B. Farabaugh, «OMG data-distribution service (DDS): architectural update,» 2004. [En línea]. Available: <https://ieeexplore.ieee.org/document/1494965>. [Último acceso: Junio 2025].
- [27] DDS-Foundation, «What is DDS?,» 2019. [En línea]. Available: <https://www.dds-foundation.org/what-is-dds-3/>. [Último acceso: Junio 2025].
- [28] O. Ozturk, «Introduction to XMPP protocol and developing online collaboration applications using open source software and libraries,» 2010. [En línea]. Available: <https://ieeexplore.ieee.org/document/5478530>. [Último acceso: Junio 2025].
- [29] Designveloper, «What is XMPP? A Complete Guide to Understanding the Messaging Protocol,» 2024. [En línea]. Available: <http://dsvgroup.medium.com/what-is-xmpp-a-complete-guide-to-understanding-the-messaging-protocol-206f3456971a>. [Último acceso: Junio 2025].
- [30] Hivemq, «MQTT Essentials,» 2025. [En línea]. Available: <https://www.hivemq.com/blog/mqtt-essentials-part-3-client-broker-connection-establishment/>. [Último acceso: Junio 2025].
- [31] HiveMQ, «MQTT Packets: A Comprehensive Guide,» 2024. [En línea]. Available: <https://www.hivemq.com/blog/mqtt-packets-comprehensive-guide/>. [Último acceso: Junio 2025].
- [32] M. Olivares, «El troviscal, pionero en el uso de Compobox,» 2020. [En línea]. Available: <https://www.canalextramadura.es/noticias/extremadura/el-torviscal-pionero-en-el-uso-de-compobox>. [Último acceso: Junio 2025].
- [33] EuropaPress, «El Consorcio MásMedio apuesta por el compostaje como solución a la gestión de residuos orgánicos municipales,» 2023. [En línea]. Available: <https://www.europapress.es/extremadura/noticia-consorcio-masmedio-apuesta-compostaje-solucion-gestion-residuos-organicos-municipales-20231005132451.html>. [Último acceso: Junio 2025].
- [34] Compostaenred, «13º Seminario de Composta en Red. Cáceres 2024,» 2024. [En línea]. Available: <https://www.compostaenred.org/seminarios-de-composta-en-red/13o-seminario-de-composta-en-red-caceres-2024>.

- [35] Retema, «El nuevo Plan de Residuos de Extremadura elimina los "macrovertederos",» 2024. [En línea]. Available: <https://www.retema.es/actualidad/el-nuevo-plan-de-residuos-de-extremadura-elimina-los-macrovertederos>. [Último acceso: Junio 2025].
- [36] A. Gaviro, «Las obras de la primera planta de compostaje FORS de Extremadura, en Llerena, comenzarán en las próximas semanas,» 2024. [En línea]. Available: <https://cadenaser.com/extremadura/2024/11/10/las-obras-de-la-primera-planta-de-compostaje-fors-de-extremadura-en-llerena-comenzaran-en-las-proximas-semanas-ser-merida/>. [Último acceso: Junio 2025].
- [37] PROYECTOS AGILES, «Desarrollo iterativo e incremental,» 2015. [En línea]. Available: <https://proyectosagiles.org/desarrollo-iterativo-incremental/>. [Último acceso: Junio 2025].
- [38] Espressif, «Espressif,» 2016. [En línea]. Available: <https://www.espressif.com/en/products/socs/esp32>. [Último acceso: Junio 2025].
- [39] I. Diestro, «Github,» 2025. [En línea]. Available: <https://github.com/idiestro/TFM-IDG-SmartComposters.git>. [Último acceso: Junio 2025].
- [40] W3Schools, «C++ Introduction,» 2019. [En línea]. Available: https://www.w3schools.com/cpp/cpp_intro.asp.
- [41] MDN Web Docs, «JavaScript,» 2025. [En línea]. Available: <https://developer.mozilla.org/es/docs/Web/JavaScript>. [Último acceso: Junio 2025].
- [42] W3Schools, «Bash Tutorial,» 2021. [En línea]. Available: <https://www.w3schools.com/bash/>. [Último acceso: Junio 2025].
- [43] MQTT, «MQTT: The Standard for IoT Messaging,» 2020. [En línea]. Available: <https://mqtt.org/>. [Último acceso: Junio 2025].
- [44] OASIS, «Oasis,» 2023. [En línea]. Available: <https://www.oasis-open.org/>. [Último acceso: Junio 2025].
- [45] MDN Web Docs, «MDN Web Docs,» 2025. [En línea]. Available: <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>. [Último acceso: Junio 2025].
- [46] European Commision, «European Commision,» 2017. [En línea]. Available: <https://digital-strategy.ec.europa.eu/en/news/fiware-european-success-story>. [Último acceso: Junio 2025].
- [47] Wikipedia, «Fiware,» 2021. [En línea]. Available: <https://es.wikipedia.org/wiki/FIWARE>. [Último acceso: Junio 2025].

- [48] J. Marschinske, «What the Hell Is Flow-Based Programming?,» 2018. [En línea]. Available: <https://medium.com/bitspark/what-the-hell-is-flow-based-programming-d9e88a6a7265>. [Último acceso: Junio 2025].
- [49] OpenJs Foundation, «About the OpenJS Foundation,» 2019. [En línea]. [Último acceso: Junio 2025].
- [50] Node-Red, «About,» 2013. [En línea]. Available: <https://nodered.org/about/>. [Último acceso: Junio 2025].
- [51] Wikipedia, «Docker,» 2020. [En línea]. Available: [https://es.wikipedia.org/wiki/Docker_\(software\)](https://es.wikipedia.org/wiki/Docker_(software)). [Último acceso: Junio 2025].
- [52] Docker, «DockerDocs,» 2015. [En línea]. Available: <https://docs.docker.com/compose/>. [Último acceso: Junio 2025].
- [53] Platformio, «What is platformio,» 2016. [En línea]. Available: <https://docs.platformio.org/en/latest/what-is-platformio.html>. [Último acceso: Junio 2025].
- [54] Microsoft, «The open source AI code editor,» 2015. [En línea]. Available: <https://code.visualstudio.com/>. [Último acceso: Junio 2025].
- [55] Eclipse, «Eclipse Mosquitto,» 2025. [En línea]. Available: <https://mosquitto.org/>. [Último acceso: Junio 2025].
- [56] Cedalo, «MQTT Broker: Pro Edition forMosquitto,» 2021. [En línea]. Available: <https://cedalo.com/mqtt-broker-pro-mosquitto/>. [Último acceso: Junio 2025].
- [57] Wikipedia, «Tipologías de Red,» 2023. [En línea]. Available: https://es.wikipedia.org/wiki/Topolog%C3%ADas_de_red. [Último acceso: Junio 2025].
- [58] NGSI-LD, «NGSI-LD,» 2025. [En línea]. Available: <https://ngsi-ld.org/>. [Último acceso: Junio 2025].
- [59] M5Stack, «Unit ENV-IV,» 2024. [En línea]. Available: <https://docs.m5stack.com/en/unit/ENV%E2%85%A3%20Unit>. [Último acceso: Junio 2025].
- [60] Autodesk Instructables, «ENS160 + AHT21 Sensor for Arduino,» 2021. [En línea]. Available: <https://www.instructables.com/ENS160-AHT21-Sensor-for-Arduino/>. [Último acceso: Junio 2025].
- [61] Sigma Robotics, «ENS160 AHT21 Air Quality Measurement Module,» 2024. [En línea]. Available: <https://store.siqma.com/ens160-aht21-air-qaulity-sensor-module.html>. [Último acceso: Junio 2025].

- [62] Electronation, «DS18B20 Temperature Sensor,» 2022. [En línea]. Available: <https://electronation.pk/shop/arduino-iot-boards/sensors/temperature-humidity/ds18b20-temperature-sensor-probe-type/>. [Último acceso: Junio 2025].
- [63] Dallas Semiconductor, «DS18B20Programmable Resolution1-Wire®Digital Thermometer,» 2017.
- [64] EPA, «What is Ph,» 2024. [En línea]. Available: <https://www.epa.gov/goldkingmine/what-ph>. [Último acceso: Junio 2025].
- [65] Amazon, «Módulo de Ph,» 2025. [En línea]. Available: https://www.amazon.es/dp/B0B4WQ44QN?ref=ppx_yo2ov_dt_b_fed_asin_title. [Último acceso: Junio 2025].
- [66] Raspberry, «Raspberry PI 5,» 2023. [En línea]. Available: <https://www.raspberrypi.com/products/raspberry-pi-5/>. [Último acceso: Junio 2025].
- [67] Lidera Energía, «¿Cuántas horas de sol al año hay en cada provincia de España?,» 2024. [En línea]. Available: <https://www.lideraenergia.com/cuantas-horas-de-sol-al-ano-hay-en-cada-provincia-de-espana/>. [Último acceso: Junio 2025].
- [68] Wikipedia, «JSON,» 2021. [En línea]. Available: <https://es.wikipedia.org/wiki/JSON>. [Último acceso: Junio 2025].
- [69] Json, «Json-es,» 2024. [En línea]. Available: <https://www.json.org/json-es.html>. [Último acceso: Junio 2025].
- [70] MQTT, «MQTT: The Standard for IoT Messaging,» 2020. [En línea]. Available: <https://mqtt.org/>. [Último acceso: Junio 2025].
- [71] A. Shalimov, «IoT in Agriculture: 9 Technology Use Cases for Smart Farming (and Challenges to Consider),» 2023. [En línea]. Available: <https://easternpeak.com/blog/iot-in-agriculture-technology-use-cases-for-smart-farming-and-challenges-to-consider/>. [Último acceso: Junio 2025].

10 Anexos

Anexo 1

Fichero de cabeceras de la librería *MeasuresReader* para la lectura de sensores.

```
/*  
  
    MeasuresReader.h - Library for get measures from composter  
sensors  
  
    Created by Ignacio Diestro Gil - 2025  
  
*/  
  
#ifndef MEASURES_READER_H  
#define MEASURES_READER_H  
  
#include <stdlib.h>  
#include <time.h>  
#include <Arduino.h>  
  
class MeasuresReader {  
    public:  
        MeasuresReader();  
        void initSensors();  
        float getEnvironmentTemperature();  
        float getEnvironmentHumidity();  
        float getEnvironmentGasO2();  
        float getEnvironmentGasCO2();  
        float getBiomassTemperature();  
        float getBiomassWeight();  
        float getBiomassFluidAmount();  
        float getBiomassPh();  
  
    private:  
        void initI2CWire();
```

```
void initEnvIv();  
void initEns160();  
void initDS18B20();  
};  
  
#endif
```

Anexo 2

Fichero de código de la librería *MeasuresReader* para la lectura de sensores.

```
/*
    MeasuresReader.cpp - Library for get measures from composter
    sensors
    Created by Ignacio Diestro Gil - 2025
*/

#include "MeasuresReader.h"

#include <stdlib.h>
#include <time.h>
#include <Arduino.h>
#include <Wire.h>
#include "M5UnitENV.h"
#include "ScioSense_ENS160.h"
#include <OneWire.h>
#include <DallasTemperature.h>

//Temporization data
unsigned long lastEns160RequestTime = 0;
//Enviv Unit
SHT4X sht4;
BMP280 bmp;
//ENS160
ScioSense_ENS160 ens160(ENS160_I2CADDR_1);
float lastCo2Value = -1.0;
bool ens160MeasurementRequested = false;
//DE18B20
#define ONE_WIRE_BUS 0
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);
```

```

//PH sensor
#define PH_PIN 35

MeasuresReader::MeasuresReader() {
}

void MeasuresReader::initSensors() {
    //Init I2C wire
    initI2CWire();
    //Init ENV IV sensor
    initEnvIv();
    //Init ENS160
    initEns160();
    //Init DS18B20
    initDS18B20();
}

// Get environment temperature from SHT4X sensor
float MeasuresReader::getEnvironmentTemperature() {
    sht4.update();
    return sht4.cTemp;
}

// Get environment humidity from SHT4X sensor
float MeasuresReader::getEnvironmentHumidity() {
    sht4.update();
    return sht4.humidity;
}

// Get environment gas O2 level
float MeasuresReader::getEnvironmentGasO2() {
    return -1;
}

```

```

}

// Get environment gas CO2 level using ENS160
float MeasuresReader::getEnvironmentGasCO2() {
    //Get measurement delay time
    const unsigned long MEASUREMENT_DELAY = 200;

    if (!ens160MeasurementRequested) {
        // New measurement requested
        ens160.measure();
        lastEns160RequestTime = millis();
        ens160MeasurementRequested = true;
    } else if (millis() - lastEns160RequestTime >=
MEASUREMENT_DELAY) {
        if (ens160.available()) {
            lastCo2Value = ens160.geteCO2();
            //Set for new meditation
            ens160MeasurementRequested = false;
        }
    }
    return lastCo2Value;
}

// Get biomass temperature from DS18B20 sensor
float MeasuresReader::getBiomassTemperature() {
    sensors.requestTemperatures();
    return sensors.getTempCByIndex(0);
}

// Get biomass weight
float MeasuresReader::getBiomassWeight() {
    return -1;
}

```

```

// Get biomass fluid amount
float MeasuresReader::getBiomassFluidAmount() {
    return -1;
}

// Get biomass pH level from analog pin
float MeasuresReader::getBiomassPh() {
    int analogValue = analogRead(PH_PIN);

    float voltage = analogValue * (5.0 / 4095.0); // 5 for 5V
    reference, could be 3.3 for 3.3V reference

    return voltage;
}

//Init I2C wire
void MeasuresReader::initI2CWire(){
    Wire.begin(32, 33);
}

//Init ENVIV UNIT with I2C
void MeasuresReader::initEnvIv(){
    //SHT40
    if (!sht4.begin(&Wire, SHT40_I2C_ADDR_44, 32, 33,
400000U)) {
        Serial.println("SHT4x no encontrado");
        while (1) delay(1);
    }

    // Precision and heater settings
    sht4.setPrecision(SHT4X_HIGH_PRECISION);
    sht4.setHeater(SHT4X_NO_HEATER);

    if (!bmp.begin(&Wire, BMP280_I2C_ADDR, 32, 33, 400000U)) {

```

```

        Serial.println("BMP28 no encontrado0");
        while (1) delay(1);
    }
    // Default settings from datasheet
    bmp.setSampling(BMP280::MODE_NORMAL,      // Operating Mode
                   BMP280::SAMPLING_X2,      // Temp. oversampling
                   BMP280::SAMPLING_X16,     // Pressure
oversampling
                   BMP280::FILTER_X16,      // Filtering
                   BMP280::STANDBY_MS_500); // Standby time
}

//Init ENS160+AHT21
void MeasuresReader::initEns160(){
    // ENS160
    if (!ens160.begin()) {
        Serial.println("ENS160 no encontrado");
        while (1);
    }
    //Set in Standard mode
    ens160.setMode(ENS160_OPMODE_STD);
    //Active automatic meditations
    //ens160.measure(true); //Not needed, batery save
}

//Init DS18B20
void MeasuresReader::initDS18B20(){
    // Initialize the OneWire bus
    sensors.begin();
    sensors.setResolution(12); // Set resolution to 12 bits
}

```

Anexo 3

Fichero de cabeceras de la librería *MqttConnection* para la conexión Wifi y MQTT.

```
/*
    MqttConnection.h - Header for mqtt transfer protocol
    Created by Ignacio Diestro Gil - 2025
*/

#ifndef MQTTCONNECTION_h
#define MQTTCONNECTION_h

#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>

class MqttConnection {
private:
    String clientId;
    WiFiClient espClient;
    PubSubClient client;
    unsigned long lastReconnectAttempt = 0;
    const unsigned long reconnectInterval = 5000; // 5 sec
    const unsigned long keepAliveInterval = 15;    // 60 sec

    void wifiConnection();
    void brokerConnection();

public:
    // Constructor
    MqttConnection();

    void init();
    void reconnect();
};
```

```
void topicSubscription();
void topicPublication(String payload);
void setClientId();
void loop();
void handleCallback(char* topic, byte* payload, unsigned int
length);

// Getter for mqtt client
PubSubClient getMqttClient();
String getClientId();

};

#endif
```

Anexo 4

Fichero de código de la librería *MqttConnection* para la conexión Wifi y MQTT

```
/*
    MqttConnection.cpp - Library for mqtt transfer protocol
    Created by Ignacio Diestro Gil - 2025
*/

#include "MqttConnection.h"
#include "LogsUtils.h"
#include "ConfigReader.h"
#include "config/mqttConnectionConfig.h"
#include "config/wifiConfig.h"

#include <WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>

//Definitions
#define payloadBufferSize 1024
MqttConnection* globalMqttInstance = nullptr;

//Callback function outside class
void mqttCallback(char* topic, byte* payload, unsigned int length)
{
    if (globalMqttInstance != nullptr) {
        globalMqttInstance->MqttConnection::handleCallback(topic,
payload, length);
    }
}

//Declarations
WiFiClient espClient;
```

```

PubSubClient client(espClient);
LogsUtils logsUtils;

//BROKER connection
String clientId;

MqttConnection::MqttConnection() : client(espClient) {
    //Set clientId
    setClientId();
}

//Setup function to init wifi and broker connection
void MqttConnection::init(){
    reconnect();
}

//Reconnect main function
void MqttConnection::reconnect(){
    //Wifi disconnection check
    if(WiFi.status() != WL_CONNECTED){
        wifiConnection();
    }
    //Broker disconnection check
    if(!client.connected()){
        //Init server
        client.setServer(MQTT_BROKER, MQTT_PORT);
        globalMqttInstance = this;
        client.setCallback(mqttCallback);
        client.setKeepAlive(keepAliveInterval);
        client.setBufferSize(payloadBufferSize);
        //Init broker connection
        brokerConnection();
        //Topic subscription

```

```

        topicSubscription();
    }
}

//Wifi connection manager
void MqttConnection::wifiConnection() {
    delay(10);
    //Print wifi connection info
    LogsUtils::println("Conectando a:");
    LogsUtils::println(WIFI_SSID);
    //Connect to Wifi
    WiFi.begin(WIFI_SSID,
ConfigReader::base64Decode(String(WIFI_PASS)));
    //Wait until successfull connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        LogsUtils::println(".");
    }
    //Show connection info
    LogsUtils::println("Conectado a Wifi");
    LogsUtils::println("IP address: " +
WiFi.localIP().toString());
}

//Mqtt initial connection
void MqttConnection::brokerConnection(){
    //Connection loop
    while (!client.connected()) {
        LogsUtils::println("Conectando al Broker...");
        //Attempt to connect
        if (client.connect(clientId.c_str())) {
            LogsUtils::println("Conectado");
            LogsUtils::println("M5Stack Tough ID: ");

```

```

        LogsUtils::printLog(clientId);
    }
    else {
        LogsUtils::printLog("Conexión fallida, rc=");
        LogsUtils::printLog(String(client.state()));
        LogsUtils::printLog(" reintentar en 5 segundos");
        // Wait 5 seconds before retrying
        delay(5000);
    }
}

}

//Subscribe to topic
void MqttConnection::topicSubscription() {
    //Topic Subscription
    if (client.subscribe(MQTT_SUBSCRIPTION_TOPIC)) {
        LogsUtils::printLog("Suscrito a topic: ");
        LogsUtils::printLog(MQTT_SUBSCRIPTION_TOPIC);
    } else {
        LogsUtils::printLog("Error al suscribirse al topic: ");
        LogsUtils::printLog(MQTT_SUBSCRIPTION_TOPIC);
    }
}

//Publish into topic
void MqttConnection::topicPublication(String payload){
    //Check if payload > buffer size
    if (payload.length() + 1 > payloadBufferSize) {
        LogsUtils::printLog("Error: El payload excede el tamaño
del buffer.");
        return;
    }

    //Parse payload into char array

```

```

char message[payloadBufferSize];
payload.toCharArray(message, payloadBufferSize);

//Send message
if (client.publish(MQTT_PUBLICATION_TOPIC, message)) {
    LogsUtils::printLog("Mensaje Mqtt");
    Serial.println(payload);
    //LogsUtils::printLog(payload);
    LogsUtils::printLog("Publicado en topic: ");
    LogsUtils::printLog(MQTT_PUBLICATION_TOPIC);
} else {
    LogsUtils::printLog("Error al publicar mensaje");
}
}

//Callback para mensajes recibidos
void MqttConnection::handleCallback(char* topic, byte* payload,
unsigned int length) {
    LogsUtils::printLog("Mensaje recibido en topic: ");
    LogsUtils::printLog(topic);
    LogsUtils::printLog("- Mensaje: ");

    String receivedMessage;
    for (int i = 0; i < length; i++) {
        receivedMessage += (char)payload[i];
    }
    LogsUtils::printLog(receivedMessage);
}

//Extract M5Tough MacAddress
void MqttConnection::setClientId() {
    clientId = WiFi.macAddress();
}

```

```

//Loop for mqtt connection
void MqttConnection::loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();
}

//Getter & Setters
PubSubClient MqttConnection::getMqttClient() {
    return client;
}

String MqttConnection::getClientId() {
    return clientId;
}

```

Anexo 5

Fichero de cabeceras de la librería *MeasuresDataHelper*, librería de ayuda para cargar las lecturas de los sensores en el json de la carga útil.

```
/*  
    MeasuresDataHelper.h - Library for set measures into Json  
    payload to send  
    Created by Ignacio Diestro Gil - 2025  
*/  
  
#ifndef MEASURES_DATA_HELPER_H  
#define MEASURES_DATA_HELPER_H  
  
#include <stdlib.h>  
#include <time.h>  
#include <Arduino.h>  
  
class MeasuresDataHelper {  
    public:  
        MeasuresDataHelper();  
        String setMeasuresIntoJson();  
        void setComposterId(String composterId);  
};  
  
#endif
```

Anexo 6

Fichero de código de la librería *MeasuresDataHelper*, librería de ayuda para cargar las lecturas de los sensores en el json de la carga útil.

```
/*
    MeasuresDataHelper.cpp - Library for set measures into Json
    payload to send
    Created by Ignacio Diestro Gil - 2025
*/

#include "MeasuresDataHelper.h"

#include <SimulatedMeasures.h>
#include <MeasuresReader.h>
#include <JsonUtils.h>
#include <data_payloads/fiwareDefaultFormatPayload.h>

#include <stdlib.h>
#include <time.h>
#include <Arduino.h>

//Init classes
SimulatedMeasures simulatedMeasures;
MeasuresReader measuresReader;
JsonUtils jsonUtils;

MeasuresDataHelper::MeasuresDataHelper() {
    //Load initial nitial Json payload
    jsonUtils.loadDefaultJson(fiwareFormatPayload);
}

// Set composter ID into the JSON payload
```

```

void MeasuresDataHelper::setComposterId(String composterId) {
    // Set composter ID
    jsonUtils.setValue("id", composterId, "string");
}

// Get measures and set into JSON payload
String MeasuresDataHelper::setMeasuresIntoJson() {
    // Set values in JSON
    //Environment variables
    jsonUtils.setValue("environmentGasCO2.value",
String(measuresReader.getEnvironmentGasCO2()), "float");
    jsonUtils.setValue("environmentGasO2.value", String(-1),
"float");
    jsonUtils.setValue("environmentHumidity.value",
String(measuresReader.getEnvironmentHumidity()), "float");
    jsonUtils.setValue("environmentTemperature.value",
String(measuresReader.getEnvironmentTemperature()), "float");
    //Biomass variables
    jsonUtils.setValue("biomassFluidAmount.value", String(-1),
"float");
    jsonUtils.setValue("biomassPh.value", String(-1), "float");
    jsonUtils.setValue("biomassTemperature.value",
String(measuresReader.getBiomassTemperature()), "float");
    jsonUtils.setValue("biomassWeight.value", String(-1),
"float");
    jsonUtils.setValue("m5BatoryLevel.value", String(-1),
"int");

    return jsonUtils.getJsonAsString();
}

```

Anexo 7

Fichero de cabeceras de la librería *LogsUtil* para mostrar logs tanto en puerto serie como en pantalla LCD.

```
/*
    LogsUtils.H - Library for logs into lcd for M5Tough
    Created by Ignacio Diestro Gil - 2025
*/

#ifndef LOGS_UTILS_H
#define LOGS_UTILS_H

#include <M5Unified.h>

class LogsUtils {
public:
    static void printLog(String message);

private:
    static int line;
    static const int lineHeight = 20;
    static const int maxLines = 10;
};

#endif
```

Anexo 8

Fichero de código de la librería *LogsUtil* para mostrar logs tanto en puerto serie como en pantalla LCD.

```
/*
    LogsUtils.cpp - Library for logs into lcd for M5Tough
    Created by Ignacio Diestro Gil - 2025
*/

#include "LogsUtils.h"

#include <M5Unified.h>

//Line counter
int LogsUtils::line=0;

/*
 * Print a message to the serial port and LCD screen.
 * If the LCD is full, it clears the screen and starts from the
 * top.
 * @param message: The message to print.
 */
void LogsUtils::printLog(String message) {
    //Print into serial port
    Serial.println(message);

    //Clean LCD if empty
    if (line >= maxLines || line == 0) {
        M5.Lcd.fillScreen(BLACK);
        M5.Lcd.setTextSize(2);
        M5.Lcd.setTextColor(WHITE);
        line = 0;
    }
}
```

```
}

    M5.Lcd.setCursor(10, 10 + line * lineHeight); // X, Y
coordinates
    M5.Lcd.print(message);
    line++;
}
```

Anexo 9

Fichero de cabeceras de la librería *JsonUtils*, librería auxiliar para trabajar con formatos Json.

```
/*
    JsonUtils.h - Library for json utilities
    Created by Ignacio Diestro Gil - 2025
*/

#ifndef JSON_UTILS_H
#define JSON_UTILS_H

#include <ArduinoJson.h>

class JsonUtils {
public:
    // Constructor
    JsonUtils();

    bool loadDefaultJson(String inputJsonAsString);

    bool setValue(const String& jsonPath, const String& newValue,
const String& valueType);

    String getJsonAsString();

private:
    DynamicJsonDocument jsonDoc;

    bool parseAndSet(JsonVariant& target, const String& value,
const String& type);
};

#endif
```

Anexo 10

Fichero de código de la librería *JsonUtils*, librería auxiliar para trabajar con formatos Json.

```
/*
    JsonUtils.cpp - Library for json utilities
    Created by Ignacio Diestro Gil - 2025
*/

#include "JsonUtils.h"

JsonUtils::JsonUtils() : jsonDoc(2048) {
}

/*
 * Load default json payload in a .h file.
 * @param filePath: Path to the JSON file.
 */
bool JsonUtils::loadDefaultJson(String inputJsonAsString) {
    // Deserialize json payload
    DeserializationError error = deserializeJson(jsonDoc,
inputJsonAsString);

    if (error) {
        Serial.print("Error al parsear el JSON: ");
        Serial.println(error.c_str());
        return false;
    }

    return true;
}

/*
```

```

* Set a new value in the JSON document at the specified path.
* @param jsonPath: Path to the JSON element (e.g.,
"root.child.grandchild").
* @param newValue: New value to set.
* @param valueType: Type of the new value (e.g., "string", "int",
"float", "bool").
*/

bool JsonUtils::setValue(const String& jsonPath, const String&
newValue, const String& valueType) {
    JsonVariant current = jsonDoc.as<JsonVariant>();
    int start = 0, end;
    while ((end = jsonPath.indexOf('.', start)) != -1) {
        String key = jsonPath.substring(start, end);
        current = current[key];
        if (current.isNull()) return false;
        start = end + 1;
    }

    String lastKey = jsonPath.substring(start);
    JsonVariant target = current[lastKey];

    return parseAndSet(target, newValue, valueType);
}

/*
* Parse a value and set it in the JSON document.
* @param target: JsonVariant to set the value in.
* @param value: Value to set.
* @param type: Type of the value (e.g., "string", "int", "float",
"bool").
*/

bool JsonUtils::parseAndSet(JsonVariant& target, const String&
value, const String& type) {
    if (type == "string") {

```

```

        target.set(value);
    } else if (type == "float") {
        target.set(value.toFloat());
    } else if (type == "int") {
        target.set(value.toInt());
    } else if (type == "bool") {
        target.set(value == "true" || value == "1");
    } else {
        Serial.println("Tipo no soportado en parseAndSet");
        return false;
    }
    return true;
}

/*
 * Get the JSON document as a string.
 * @return: JSON document as a string.
 */
String JsonUtils::getJsonAsString() {
    String output;
    serializeJson(jsonDoc, output);
    return output;
}

```

Anexo 11

Fichero de cabeceras de la librería *ConfigReader*, librería para lectura de ficheros encriptados de configuración.

```
/*
    ConfigReader.h - Library for read crypted properties
    Created by Ignacio Diestro Gil - 2025
*/

#ifndef CONFIG_READER_H
#define CONFIG_READER_H

#include <ArduinoJson.h>

class ConfigReader {
public:
    // Constructor
    ConfigReader();

    static const char* base64Decode(const String &encoded);

private:
    static bool isBase64(unsigned char c);
};

#endif
```

Anexo 12

Fichero de código de la librería *ConfigReader*, librería para lectura de ficheros encriptados de configuración.

```
/*
    ConfigReader.cpp - Library for read crypted properties
    Created by Ignacio Diestro Gil - 2025
*/

#include "ConfigReader.h"

#include <Arduino.h>
#include <stdlib.h>

ConfigReader::ConfigReader(){
}

/*
 * Decrypt a base64 encoded string.
 * @param encoded: Base64 encoded string.
 * @return: Decrypted string.
 */
const char* ConfigReader::base64Decode(const String &encoded) {
    const char* base64_chars =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        "abcdefghijklmnopqrstuvwxyz"
        "0123456789+/";

    int in_len = encoded.length();
    int i = 0, in_ = 0;
    uint8_t char_array_4[4], char_array_3[3];
    int output_len = (in_len * 3) / 4;
```

```

    char* output = (char*)malloc(output_len + 1); // +1 para el
    null terminator

    int pos = 0;

    if (!output) return nullptr;

    while (in_len-- && (encoded[in_] != '=')) &&
    isBase64(encoded[in_])) {
        char_array_4[i++] = encoded[in_]; in_++;
        if (i == 4) {
            for (i = 0; i < 4; i++)
                char_array_4[i] = strchr(base64_chars,
                char_array_4[i]) - base64_chars;

            char_array_3[0] = (char_array_4[0] << 2) +
            ((char_array_4[1] & 0x30) >> 4);
            char_array_3[1] = ((char_array_4[1] & 0xf) << 4) +
            ((char_array_4[2] & 0x3c) >> 2);
            char_array_3[2] = ((char_array_4[2] & 0x3) << 6) +
            char_array_4[3];

            for (i = 0; i < 3; i++) output[pos++] =
            (char)char_array_3[i];
            i = 0;
        }
    }

    if (i) {
        for (int j = i; j < 4; j++)
            char_array_4[j] = 0;

        for (int j = 0; j < 4; j++)
            char_array_4[j] = strchr(base64_chars,
            char_array_4[j]) - base64_chars;
    }

```

```

        char_array_3[0] = (char_array_4[0] << 2) +
        ((char_array_4[1] & 0x30) >> 4);

        char_array_3[1] = ((char_array_4[1] & 0xf) << 4) +
        ((char_array_4[2] & 0x3c) >> 2);

        char_array_3[2] = ((char_array_4[2] & 0x3) << 6) +
        char_array_4[3];

        for (int j = 0; j < (i - 1); j++) output[pos++] =
        (char)char_array_3[j];
    }

    output[pos] = '\0'; // Null-terminator
    return output;
}

bool ConfigReader::isBase64(unsigned char c) {
    return (isalnum(c) || (c == '+') || (c == '/'));
}

```

Anexo 13

Fichero *docker-compose.yml*.

version: "3.8"

services:

mosquitto:

image: eclipse-mosquitto:2

container_name: mosquitto

restart: unless-stopped

ports:

- "1884:1884"

volumes:

- ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf

- ./mosquitto/data:/mosquitto/data

networks:

- iot-net

node-red:

image: nodered/node-red:latest

container_name: node-red

restart: unless-stopped

ports:

- "1880:1880"

volumes:

- ./node-red/data:/data

networks:

- iot-net

depends_on:

- mosquitto

networks:

iot-net:

driver: bridge

Anexo 14

Código JS para función personalizada en *Node-Red* 1.

```
// Almacenar el Id de la compostera
global.set('composter-id', msg.payload.id);

//Eliminar Id y type del payload
delete msg.payload.id;
delete msg.payload.type;
// Almacenar el payload en el contexto de flujo
global.set('mqtt-payload', msg.payload);
```

Anexo 15

Código JS para función personalizada en *Node-Red*. Envío de datos a Fiware

```
// Headers
msg.headers = {
  'Content-Type': 'application/json',
  'Fiware-Service': 'smartprovincia',
  'Fiware-ServicePath': '/Compostera',
  'X-Auth-Token': flow.get('auth_token'),

};

// Body/Payload
msg.payload = global.get('mqtt-payload');

// URL
const composterId = global.get('composter-id')
msg.url = `https://cb.iotplatform.telefonica.com:10027/v2/entities/${compo
sterId}/attrs`;

// MSG
return msg;
```

Anexo 16

Código JS para función personalizada en *Node-Red* 3. Envío de datos a Fiware, solicitud de token.

```
// Headers
msg.headers = {
  'Content-Type': 'application/json' // Configura el Content-
  Type
};

// Payload
msg.payload = {
  "auth": {
    "identity": {
      "methods": [
        "password"
      ],
      "password": {
        "user": {
          "domain": {
            "name": "smartprovincia"
          },
          "name": "userCompostera",
          "password": "badajozesmas"
        }
      }
    },
    "scope": {
      "project": {
        "domain": {
          "name": "smartprovincia"
        },

```

```
        "name": "/Compostera"
    }
}
};

// MSG
return msg;
```